

AI 생성 도서

파이썬으로 배우는 머신러닝

파이썬 기초 문법은 알고 있지만 머신러닝은 처음인 독자를 위한 입문서.
복잡한 수학보다는 직관적인 비유와 실습 중심으로 머신러닝을 설명하며,
독자가 직접 데이터를 분석하고 머신러닝 모델을 학습시킬 수 있도록 구성합니다.

AI Book Generator · 2026-06-18

gemma4:31b · 버전 2

파이썬으로 배우는 머신러닝

파이썬 기초 문법은 알고 있지만 머신러닝은 처음인 독자를 위한 입문서. 복잡한 수학보다는 직관적인 비유와 실습 중심으로 머신러닝을 설명하며, 독자가 직접 데이터를 분석하고 머신러닝 모델을 학습시킬 수 있도록 구성합니다.

발행일 2026-06-18

지은이 AI Book Generator

생성 모델 gemma4:31b

목차 구성 지정 목차

작성 소요 시간 5h 56m 18s

버전 v2

본 문서는 자동 생성 파이프라인으로 작성·검수되었습니다.

© 2026 AI Book Generator. All rights reserved.

목차

1. 머신러닝이란 무엇인가	8	이번 장에서 배우는 내용	8
		실생활 비유: 요리 레시피 전수하기	8
		개념 설명: 전통적인 프로그래밍 vs 머신러닝	8
		기술 설명: AI, 머신러닝, 딥러닝의 관계	9
		머신러닝이 왜 필요한가? (실생활 사례)	9
		파이썬으로 맛보는 머신러닝의 흐름	10
		머신러닝의 학습 방식: 지도, 비지도, 강화학습	11
		입문자가 흔히 하는 오해와 실수	12
		핵심 요약	12
		확인 문제	13
		정답 및 해설	13
		미니 프로젝트: 나만의 간단한 분류기 만들기	13
2. 파이썬 머신러닝 개발 환경 구축	14	이번 장에서 배우는 내용	14
		실생활 비유: 레스토랑 주방 vs 테스트 키친	14
		개념 설명: 왜 '대화형 환경'이 필요한가?	14
		기술 설명	15
		파이썬 코드 실습	16
		실행 결과 해설	18
		실무 활용 사례	18
		자주 하는 실수	18
		핵심 요약	19
		확인 문제	19
		정답 및 해설	19
		미니 프로젝트: 나만의 분석 환경 리포트 만들기	20
3. NumPy와 Pandas			21

4. 데이터 시각화	22	이번 장에서 배우는 내용	22
		실생활 비유: 엑셀 시트와 사진의 차이	22
		데이터 시각화의 필요성: 왜 그래프를 그려야 하는가?	22
		Matplotlib: 데이터 시각화의 뼈대 만들기	23
		Seaborn: 더 쉽고 아름답게 데이터 분석하기	24
		데이터의 성격에 맞는 그래프 선택법	25
		실무 활용 사례: 데이터 탐색적 분석(EDA)의 흐름	26
		시각화 과정에서 자주 하는 실수	27
		핵심 요약	27
		확인 문제	27
		미니 프로젝트: 타이타닉 생존자 데이터 분석하기	28
5. 데이터 전처리	29	이번 장에서 배우는 내용	29
		실생활 비유: 요리 전 '재료 손질'	29
		개념 설명: 왜 전처리가 필요한가?	30
		기술 설명: 전처리의 4대 핵심 기법	30
		파이썬 코드 실습	30
		실무 활용 사례: 고객 이탈 예측 시스템	31
		자주 하는 실수	32
		핵심 요약	32
		확인 문제	32
		정답 및 해설	33
		미니 프로젝트: 중고차 가격 예측 전처리 파이프라인 만들기	33
6. 지도학습			34
[수정본] 00. 지도 학습 (Supervised Learning)			35

7. 회귀	37	이번 장에서 배우는 내용	37
		실생활 비유: "공부 시간과 시험 점수의 상관관계"	37
		개념 설명: 회귀(Regression)란 무엇인가?	37
		기술 설명: 선형 회귀의 원리	38
		파이썬 코드 실습: Scikit-learn으로 회귀 모델 만들기	38
		모델의 성적표: 회귀 모델 평가 지표	39
		더 나은 예측을 향해: 단순 회귀에서 다중 회귀로	40
		실무 활용 사례	41
		자주 하는 실수	41
		핵심 요약	41
		확인 문제	41
		미니 프로젝트: 우리 동네 카페 매출 예측하기	42
8. 분류			43
9. 비지도학습과 군집화	44	이번 장에서 배우는 내용	44
		정답 없는 학습: 비지도학습의 개념	44
		끼리끼리 묶기: K-Means 군집화의 원리	45
		파이썬으로 구현하는 K-Means 군집화	45
		최적의 K값 찾기: 엘보우 방법(Elbow Method)	46
		비지도학습의 실무 활용 사례	48
		초보자가 자주 하는 실수와 해결책	48
		핵심 요약	48
		확인 문제	49
		정답 및 해설	49
		미니 프로젝트: 쇼핑몰 고객 분류하기	49
10. 모델 평가와 성능 개선			50

11. 과적합과 하이퍼파라미터 튜닝	51	이번 장에서 배우는 내용	51
		암기왕과 응용력: 과적합(Overfitting)의 정체	51
		유연함과 딱딱함의 균형: 편향-분산 트레이드오프	52
		모델의 설정값 조정하기: 하이퍼파라미터 튜닝	53
		과적합을 막는 안전장치: 규제(Regularization)	53
		실습: 최적의 모델을 완성하는 튜닝 프로세스	55
		초보자가 자주 하는 실수: '데이터 누수'와 '검증 데이터 오염'	57
		확인 문제	58
		정답 및 해설	58
		핵심 요약 및 미니 프로젝트	59
12. 앙상블과 실전 머신러닝	60	이번 장에서 배우는 내용	60
		집단지성의 힘: 앙상블 학습의 개념	60
		배깅(Bagging): 무작위로 샘플링하여 분산을 줄여 과적합을 방지하는 방법	61
		부스팅(Boosting): 틀린 문제에 집중하여 점수를 올리는 방법	62
		실전! 앙상블 모델로 성능 끌어올리기	63
		앙상블 모델의 해석력과 중요도	64
		실무 활용 사례	65
		앙상블 모델 사용 시 주의사항과 실수	65
		핵심 요약	65
		확인 문제	65
		미니 프로젝트: 우리 동네 집값 예측 앙상블 모델 만들기	66
13. 실전 프로젝트 - 타이타닉 생존자 예측	67	이번 장에서 배우는 내용	67
		실생활 비유: 데이터 분석가는 '디지털 탐정'이다	67
		1. 프로젝트 개요 및 데이터 탐색 (EDA)	67
		2. 데이터 전처리 및 피처 엔지니어링	68
		3. 모델 선택 및 학습	69
		4. 모델 평가 및 최적화	70
		5. 실무 활용 사례: 이진 분류 모델의 확장	71
		6. 프로젝트 마무리: 인사이트 도출 및 회고	71
		확인 문제	72
		정답 및 해설	72
		미니 프로젝트: 나만의 생존 예측 파이프라인 구축하기	72

14. 실전 프로젝트 - 집값 예측	74	이번 장에서 배우는 내용	74
		실생활 비유: 베테랑 공인중개사의 감정 평가	74
		부동산 데이터의 특성과 예측의 본질	74
		데이터 탐색 및 가설 설정 (EDA)	75
		특성 공학: 데이터의 가치를 높이는 과정	75
		파이썬 코드 실습: 집값 예측 파이프라인	76
		모델 성능 평가 및 결과 해석	79
		실전적 관점의 모델 개선 및 마무리	79
		자주 하는 실수	79
		핵심 요약	80
		확인 문제	80
		미니 프로젝트: 나만의 집값 예측 모델 고도화	80
15. 실전 프로젝트 - 영화 추천 시스템			82

1. 머신러닝이란 무엇인가

이번 장에서 배우는 내용

머신러닝이라는 단어를 들으면 보통 '똑똑한 로봇'이나 '복잡한 수학 공식'을 먼저 떠올리곤 합니다. 하지만 머신러닝의 본질은 생각보다 단순합니다. 이번 장에서는 머신러닝이 정확히 무엇인지, 그리고 우리가 기존에 해왔던 프로그래밍과는 어떻게 다른지 알아봅니다. 또한, 인공지능, 머신러닝, 딥러닝이라는 헷갈리는 용어들을 정리하고, 파이썬 코드를 통해 머신러닝이 데이터를 학습하고 예측하는 전체적인 흐름을 가볍게 경험해 보겠습니다.

실생활 비유: 요리 레시피 전수하기

여러분에게 '김치찌개 맛있게 끓이는 법'을 가르쳐준다고 가정해 봅시다. 여기에는 두 가지 방법이 있습니다.

첫 번째 방법은 아주 상세한 **레시피(규칙)**를 적어주는 것입니다. "김치 200g을 넣고, 물 500ml를 붓고, 10분간 끓인 뒤, 간장 1큰술을 넣어라." 이 방법은 명확합니다. 하지만 재료의 상태(김치의 신맛 정도)나 불의 세기가 달라지면 맛이 변하게 됩니다. 이때마다 레시피를 수정해서 다시 알려줘야 하죠.

두 번째 방법은 선생님이 끓인 **맛있는 김치찌개 100그릇(데이터와 결과)**을 직접 먹어보게 하는 것입니다. 학생은 100그릇의 맛을 보며 "아, 김치가 이 정도로 신 상태에서 이만큼 끓였을 때 가장 맛있구나!"라는 **패턴(규칙)**을 스스로 깨닫습니다. 레시피를 직접 읽지 않았지만, 수많은 사례를 통해 '맛있는 찌개'를 스스로 학습한 것입니다.

여기서 두 번째 방법이 바로 **머신러닝(Machine Learning)**의 방식입니다. 사람이 일일이 규칙을 정해주는 것이 아니라, 컴퓨터가 데이터를 통해 스스로 규칙을 찾아내게 만드는 것이죠.

개념 설명: 전통적인 프로그래밍 vs 머신러닝

우리가 지금까지 배워온 일반적인 프로그래밍은 '명시적 프로그래밍(Explicit Programming)'입니다. 개발자가 `if-else` 문이나 `for` 문을 사용해 컴퓨터가 따라야 할 규칙을 하나하나 정해주는 방식입니다.

1. 전통적인 프로그래밍 (Rule-based)

- **작동 방식:** 데이터와 규칙을 입력하면 결과가 나옵니다.
- **공식:** 데이터 + 규칙 $\rightarrow$ 결과

- **특징:** 규칙이 명확한 작업(예: 계산기, 회원가입 폼 검증)에 매우 효율적입니다. 하지만 규칙이 너무 복잡하거나 계속 변하는 작업에는 한계가 있습니다.

2. 머신러닝 (Data-driven)

- **작동 방식:** 데이터와 정답(결과)을 입력하면 컴퓨터가 그 사이의 규칙을 찾아냅니다.
- **공식:** 데이터 + 결과 $\rightarrow$ 규칙
- **특징:** 사람이 일일이 규칙을 설명하기 어려운 복잡한 패턴(예: 사진 속 고양이 찾기, 주가 예측)을 해결하는 데 강력합니다.

구분	전통적 프로그래밍	머신러닝
핵심	사람이 규칙을 정의함	컴퓨터가 규칙을 학습함
입력값	데이터, 규칙	데이터, 결과(정답)
출력값	결과	규칙(모델)
접근 방식	명시적 프로그래밍	데이터 기반 학습

기술 설명: AI, 머신러닝, 딥러닝의 관계

뉴스나 인터넷을 보면 인공지능, 머신러닝, 딥러닝이라는 말이 혼용되어 쓰입니다. 하지만 이 세 가지는 서로 다른 개념이 아니라, **포함 관계**에 있습니다.

1. **인공지능 (AI, Artificial Intelligence):** 가장 넓은 개념입니다. 인간의 지능을 모방하여 지적인 작업을 수행할 수 있게 하는 모든 기술을 통칭합니다. (예: 체스 프로그램, 단순한 챗봇 등)
2. **머신러닝 (ML, Machine Learning):** AI의 한 분야로, 명시적으로 프로그래밍되지 않고 데이터로부터 패턴을 학습하여 성능을 개선하는 알고리즘을 말합니다.
3. **딥러닝 (DL, Deep Learning):** 머신러닝의 한 종류로, 인간의 뇌 구조를 모방한 '인공신경망 (ANN)'을 깊게 쌓아 학습하는 기술입니다. 이미지 인식이나 자연어 처리(ChatGPT 등)에서 압도적인 성능을 보입니다.

[도식화] 인공지능(AI) $\supset$ 머신러닝(ML) $\supset$ 딥러닝(DL)

머신러닝이 왜 필요한가? (실생활 사례)

왜 굳이 복잡하게 머신러닝을 쓸까요? 사람이 규칙을 짜는 것이 불가능에 가까운 영역이 있기 때문입니다.

예: 스팸 메일 필터링 전통적인 방식으로 스팸 메일을 거르려면 다음과 같이 코드를 짜야 할 것입니다.

```
if "광고" in email_text or "무료" in email_text: spam = True
```

하지만 스팸 메일 발송자들은 '광고'라는 단어를 피하기 위해 `광.고`, `광 고`, `Gwang-go` 처럼 변형해서 보냅니다. 사람이 이 모든 변형 패턴을 `if` 문으로 다 작성할 수 있을까요? 불가능합니다.

머신러닝은 수만 통의 '스팸 메일'과 '정상 메일' 데이터를 학습합니다. 이때 단순히 단어를 찾는 것이 아니라, "특정 단어들이 함께 나타날 확률이 얼마나 높은지" 혹은 "발신 주소와 본문 패턴의 통계적 특성이 스팸과 얼마나 유사한지"와 같은 확률적 패턴을 스스로 학습합니다. 우리가 가르쳐주지 않은 미세한 확률적 징후까지 잡아내는 것이 머신러닝의 진짜 힘입니다.

파이썬으로 맛보는 머신러닝의 흐름

이제 이론은 접어두고, 파이썬의 대표적인 머신러닝 라이브러리인 **Scikit-learn**(사이킷런)을 사용해 아주 간단한 예측 모델을 만들어 보겠습니다.

머신러닝의 핵심 프로세스는 '학습 $\rightarrow$ 모델 생성 $\rightarrow$ 예측'의 흐름으로 이어집니다. * **학습(fit)**: 데이터 속에서 패턴을 찾아 '모델'이라는 일종의 규칙 뭉치를 만드는 과정입니다. * **예측(predict)**: 이렇게 만들어진 규칙 뭉치(모델)에 새로운 데이터를 넣어 결과를 도출하는 과정입니다.

우리는 '과일의 무게'와 '색상' 데이터를 보고, 이 과일이 '사과'인지 '오렌지'인지 맞히는 모델을 만들어 보겠습니다.

실습 코드

```
# 1. 필요한 라이브러리 불러오기
from sklearn import tree

# 2. 데이터 준비
# 특성(Feature): [무게(g), 색상(0: 빨강, 1: 주황)]
features = [[140, 0], [130, 0], [150, 1], [170, 1]]

# 정답(Label): [0: 사과, 1: 오렌지]
labels = [0, 0, 1, 1]

# 3. 모델 생성 (결정 트리 알고리즘 사용)
# 결정 트리는 스무개처럼 질문을 던져 정답을 찾아가는 알고리즘입니다.
clf = tree.DecisionTreeClassifier()

# 4. 학습 (Training)
# fit 메서드는 데이터와 정답을 통해 '모델(규칙 뭉치)'을 생성합니다.
clf.fit(features, labels)

# 5. 예측 (Prediction)
# 생성된 모델(clf)에 새로운 데이터 [무게 160g, 색상 주황(1)]를 넣어 결과를 예측합니다.
new_fruit = [[160, 1]]
prediction = clf.predict(new_fruit)

# 결과 출력
fruit_name = "오렌지" if prediction[0] == 1 else "사과"
print(f"새로운 과일의 예측 결과: {fruit_name}")
```

실행 결과 해설

- **결과:** 새로운 과일의 예측 결과: 오렌지
- **해설:**
 - `clf.fit(features, labels)` 단계에서 모델은 "무게가 140g 이하이면 빨간색이면 사과고, 150g 이상이면 주황색이면 오렌지구나!"라는 **규칙 뭉치(모델)**을 완성했습니다.
 - 그 후 `clf.predict([[160, 1]])` 를 호출하자, 모델은 자신이 가진 규칙 뭉치를 바탕으로 160g의 주황색 과일은 '오렌지'라고 판단한 것입니다.

머신러닝의 학습 방식: 지도, 비지도, 강화학습

머신러닝은 학습하는 방식에 따라 크게 세 가지로 나뉩니다. 앞으로 우리가 배울 내용들의 지도라고 생각하세요.

1. 지도학습 (Supervised Learning):

- **특징:** 문제(데이터)와 정답(레이블)을 함께 줍니다.
- **비유:** 선생님이 정답지를 주고 공부시키는 것.
- **예시:** 스팸 메일 분류, 집값 예측.

2. 비지도학습 (Unsupervised Learning):

- **특징:** 정답 없이 문제(데이터)만 줍니다. 데이터 간의 **거리나 유사도**를 측정해 비슷한 특성을 가진 것끼리 묶습니다. 이를 '**군집화(Clustering)**'라고 합니다.
- **비유:** 정답지 없이 스스로 공통점을 찾아 분류하는 것.
- **예시:** 고객 세그먼트 분류(비슷한 구매 성향을 가진 고객끼리 묶기).

3. 강화학습 (Reinforcement Learning):

- **특징:** 정답 대신 어떤 행동을 했을 때 '보상'을 줍니다. 수많은 **시행착오(Trial and Error)**를 겪으며, 보상을 최대화하기 위해 자신의 행동 전략을 계속해서 수정합니다.
- **비유:** 강아지에게 간식을 주며 훈련시키는 것.
- **예시:** 알파고(AlphaGo), 자율주행 자동차.

입문자가 흔히 하는 오해와 실수

머신러닝을 처음 접할 때 가장 위험한 생각은 "**모델만 좋으면 어떤 데이터든 마법처럼 해결해 줄 것이다**"라고 믿는 것입니다.

머신러닝 업계에는 **GIGO (Garbage In, Garbage Out)**라는 유명한 말이 있습니다. "**쓰레기가 들어가면 쓰레기가 나온다**"는 뜻입니다.

- **실수 사례:** 아주 최신 딥러닝 모델을 사용했는데 예측 정확도가 엉망인 경우.
- **원인:** 알고리즘의 문제가 아니라, 학습에 사용한 데이터에 오류가 많거나 양이 너무 적었기 때문일 확률이 높습니다.
- **교훈:** 모델의 정교함보다 더 중요한 것은 **데이터의 품질**입니다. 깨끗하고 정확한 데이터를 준비하는 것이 머신러닝 성공의 80%를 결정합니다.

핵심 요약

- 전통적 프로그래밍은 데이터 + 규칙 $\rightarrow$ 결과 이지만, 머신러닝은 데이터 + 결과 $\rightarrow$ 규칙 을 찾아내는 과정이다.
- $AI \supset \text{머신러닝} \supset \text{딥러닝}$ 순의 포함 관계를 가진다.
- 머신러닝의 기본 프로세스는 학습 (fit) $\rightarrow$ 모델(규칙 뭉치) 생성 $\rightarrow$ 예측 (predict) 순으로 이루어진다.
- 학습 방식에는 정답이 있는 지도학습, 유사도를 기반으로 묶는 비지도학습(군집화), 시행착오를 통해 보상을 최대화하는 강화학습이 있다.
- **GIGO 원칙:** 데이터의 품질이 모델의 성능을 결정한다.

확인 문제

1. 전통적인 프로그래밍과 머신러닝의 결정적인 차이점은 무엇인가요?
2. 다음 중 포함 관계가 옳은 것을 고르세요. (A) 딥러닝 $\supset$ 머신러닝 $\supset$ 인공지능
(B) 인공지능 $\supset$ 머신러닝 $\supset$ 딥러닝
3. 정답(Label)이 없는 데이터를 사용하여 데이터 간의 유사도를 측정하고 그룹을 찾아내는 학습 방식은 무엇인가요?

정답 및 해설

1. **정답:** 전통적인 프로그래밍은 사람이 직접 규칙(Rule)을 정의하여 결과를 내는 방식이지만, 머신러닝은 데이터와 결과를 통해 컴퓨터가 스스로 규칙을 찾아내는 방식입니다.
2. **정답:** (B) 인공지능이라는 큰 틀 안에 머신러닝이 있고, 머신러닝의 한 종류로 딥러닝이 존재합니다.
3. **정답:** 비지도학습(Unsupervised Learning)

미니 프로젝트: 나만의 간단한 분류기 만들기

목표: 위에서 배운 `DecisionTreeClassifier` 를 활용해 '공부 시간'과 '수면 시간'에 따른 '시험 합격 여부'를 예측하는 모델을 만들어 보세요.

• 데이터 힌트:

- 특성: [공부 시간, 수면 시간]
- 정답: [0: 불합격, 1: 합격]

• 미션:

1. 5개 이상의 데이터셋을 직접 만들어 학습시키세요.
2. 새로운 데이터(예: 공부 10시간, 수면 4시간)를 넣었을 때 합격일지 불합격일지 예측해 보세요.
3. 데이터를 수정했을 때 예측 결과가 어떻게 변하는지 관찰해 보세요.

2. 파이썬 머신러닝 개발 환경 구축

이번 장에서 배우는 내용

머신러닝은 단순히 코드를 작성하는 것이 아니라, 데이터를 관찰하고 가설을 세우며 실험하는 과정의 반복입니다. 따라서 일반적인 소프트웨어 개발과는 다른 '실험실' 같은 환경이 필요합니다. 이번 장에서는 머신러닝 학습을 위한 최적의 디지털 실험실을 구축하는 방법을 배웁니다.

- 머신러닝 개발 환경이 일반 개발 환경과 어떻게 다른지 이해합니다.
 - 파이썬을 설치하고, 프로젝트 간의 충돌을 방지하는 '가상환경'과 '커널'의 개념을 익힙니다.
 - 데이터 과학자의 필수 도구인 '주피터 노트북(Jupyter Notebook)'과 'VS Code'를 설치하고 활용법을 배웁니다.
-

실생활 비유: 레스토랑 주방 vs 테스트 키친

우리가 일반적인 앱이나 프로그램을 만드는 것은 '**레스토랑의 주방**'에서 요리하는 것과 비슷합니다. 정해진 레시피(설계도)에 따라 재료를 준비하고 요리를 완성해 손님에게 내놓으면 끝납니다. 한 번 완성된 요리는 맛이 변하지 않으며, 정해진 순서대로 정확히 작동하는 것이 가장 중요합니다.

반면, 머신러닝 개발은 '**테스트 키친(실험실)**'에서 새로운 레시피를 개발하는 과정과 같습니다. 소금을 한 꼬집 넣어보고 맛을 본 뒤, "음, 조금 더 짜야겠는데?"라고 생각하며 다시 간을 맞춥니다. 불의 세기를 조금 높여보고 다시 맛을 보는 과정을 수백 번 반복하며 최적의 맛(모델의 성능)을 찾아냅니다.

머신러닝 환경은 바로 이 '**맛을 보며 수정하는 과정**'을 효율적으로 만들어주는 도구들의 집합입니다.

개념 설명: 왜 '대화형 환경'이 필요한가?

전통적인 프로그래밍 방식은 **스크립트(Script)** 방식입니다. 메모장에 코드를 모두 작성한 뒤, 실행 버튼을 눌러 처음부터 끝까지 한 번에 결과를 확인합니다. 만약 중간에 오류가 있거나 데이터가 예상과 다르다면, 다시 수정하고 처음부터 다시 실행해야 합니다.

하지만 머신러닝은 데이터의 형태를 확인하고, 그래프를 그려보고, 모델의 파라미터를 조금씩 수정하는 작업이 핵심입니다. 이때 필요한 것이 **대화형(Interactive)** 방식입니다.

- **스크립트 방식**: "모든 재료를 넣고 냄비를 닫은 뒤, 30분 후에 뚜껑을 열어 맛을 본다." $\rightarrow$ 맛이 없으면 처음부터 다시 요리해야 함.

- **대화형 방식:** "재료를 하나 넣고 맛을 본다 $\rightarrow$ 다음 재료를 넣고 또 맛을 본다." $\rightarrow$ 어느 단계에서 문제가 생겼는지 즉시 알 수 있고, 이전 단계의 결과를 유지한 채 다음 실험을 이어갈 수 있음.

특히 머신러닝에서는 '**상태 유지(State)**'라는 개념이 매우 중요합니다. 수 기가바이트(GB)에 달하는 대용량 데이터를 읽어오는 데 5분이 걸린다고 가정해 봅시다. 스크립트 방식은 코드를 수정할 때마다 매번 5분을 기다려 데이터를 다시 읽어야 하지만, 대화형 환경에서는 **한 번 로드한 데이터를 메모리에 올려둔 상태로 유지**할 수 있습니다. 덕분에 데이터는 그대로 둔 채, 분석 코드만 수정해 즉시 결과를 확인할 수 있어 실험 속도가 비약적으로 빨라집니다.

기술 설명

1. 파이썬 설치: 기초 공사

머신러닝의 기본 언어인 파이썬을 먼저 설치해야 합니다.

- **설치 방법:** python.org 공식 홈페이지에서 최신 안정 버전을 다운로드하여 설치합니다.
- **매우 중요한 주의사항:** 설치 창이 뜨면 하단에 있는 **Add Python to PATH** 체크박스를 반드시 선택하세요. 이 설정을 하지 않으면 터미널(CMD)에서 `python` 이나 `pip` 명령어를 인식하지 못해 설치 후 실행 단계에서 오류가 발생합니다.

2. 패키지 관리자와 가상환경

머신러닝을 하려면 파이썬과 함께 수학 계산을 돕는 `NumPy`, 데이터 분석을 돕는 `Pandas`, 머신러닝 알고리즘이 구현된 `scikit-learn` 같은 **라이브러리(패키지)**가 필요합니다.

pip: 라이브러리 배달 서비스

`pip`는 파이썬의 패키지 관리자로, 인터넷에 공개된 수많은 라이브러리를 내 컴퓨터에 간단하게 설치해 주는 도구입니다. 터미널에서 `pip install 라이브러리이름` 이라고 입력하면 자동으로 다운로드되어 설치됩니다.

가상환경(Virtual Environment)과 커널(Kernel)

프로젝트 A에서는 `scikit-learn` 1.0 버전이 필요한데, 프로젝트 B에서는 2.0 버전이 필요할 수 있습니다. 파이썬은 기본적으로 하나의 버전을 설치하므로, 두 프로젝트를 동시에 진행하면 버전 충돌이 일어나 프로그램이 망가질 수 있습니다.

이를 해결하기 위해 **가상환경**을 사용합니다. 가상환경은 내 컴퓨터 안에 '**독립된 작은 방**'을 만드는 것과 같습니다. - **방 A:** 파이썬 3.9 + `scikit-learn` 1.0 - **방 B:** 파이썬 3.11 + `scikit-learn` 2.0

실제로 `python -m venv ml_env` 라는 명령어를 실행하면, 현재 작업 중인 폴더에 **ml_env** 라는 이름의 **실제 폴더**가 생성됩니다. 이 폴더가 바로 '방'의 실체이며, 전용 파이썬 실행 파일과 라이브러리들이 물리적으로 저장됩니다.

여기서 **커널(Kernel)**이라는 개념이 등장합니다. 가상환경이 라이브러리가 저장된 '방'이라면, 커널은 그 방 안에서 **사용자가 입력한 코드를 실제로 계산하여 결과를 돌려주는 '백엔드 프로세스(엔진)'**입니다. 우리가 가상환경을 만들고 커널을 등록하는 이유는, 주피터 노트북 같은 도구에게 "시스템 기본 엔진 말고, 내가 만든 이 방의 엔진(커널)을 사용해줘!"라고 지정하기 위해서입니다.

3. 주피터 노트북: 데이터 과학자의 스케치북

주피터 노트북은 '대화형 방식'을 구현한 대표적인 도구로, '**셀(Cell)**' 단위로 구성되어 있습니다.

- **코드 셀(Code Cell):** 파이썬 코드를 작성하고 실행하는 곳입니다. 결과가 셀 바로 아래에 즉시 출력됩니다.
- **마크다운 셀(Markdown Cell):** 메모, 설명, 수식, 이미지 등을 넣는 곳입니다.

이 덕분에 주피터 노트북은 단순한 코드 파일이 아니라, [가설 설정 $\rightarrow$ 코드 실행 $\rightarrow$ 결과 확인 $\rightarrow$ 해석 및 결론]의 흐름을 담은 '**분석 리포트**'가 됩니다.

4. VS Code: 현대적인 통합 개발 환경(IDE)

최근의 데이터 과학자들은 주피터 노트북의 편리함과 전문 개발 도구의 강력함을 모두 누리기 위해 **VS Code(Visual Studio Code)**를 사용합니다.

- **통합 환경:** 가상환경 관리, Git 버전 관리, 코드 자동 완성 기능을 모두 제공합니다.
- **주피터 확장 프로그램:** VS Code에서 'Jupyter' 확장 프로그램을 설치하면, 웹 브라우저를 켜지 않고도 `.ipynb` 파일을 그대로 열어 코드 셀을 실행할 수 있습니다.
- **현대적인 흐름:** 실험(주피터 노트북) $\rightarrow$ 정제(파이썬 스크립트 파일 `.py`) $\rightarrow$ 배포로 이어지는 전체 개발 흐름을 하나의 도구에서 처리할 수 있습니다.

파이썬 코드 실습

이제 실제로 머신러닝 실험실을 구축해 보겠습니다. (Windows 기준)

Step 1: 가상환경 생성 및 활성화

터미널(명령 프롬프트)을 열고 다음 명령어를 입력하세요.


```
# 1. 머신러닝 프로젝트 폴더 생성 및 이동
mkdir ml_study
cd ml_study

# 2. 'ml_env'라는 이름의 가상환경 생성
python -m venv ml_env

# 3. 가상환경 활성화
# Windows:
ml_env\Scripts\activate
# macOS/Linux:
source ml_env/bin/activate
```

Step 2: 필수 라이브러리 및 커널 등록

가상환경이 활성화된 상태(`ml_env` 표시가 뜬 상태)에서 라이브러리를 설치하고, 주피터 노트북이 이 환경을 인식할 수 있도록 커널을 등록해야 합니다.

```
# 최신 버전의 pip로 업데이트
python -m pip install --upgrade pip

# 머신러닝 필수 라이브러리 및 주피터 설치
pip install numpy pandas matplotlib scikit-learn jupyter

# [중요] 가상환경을 주피터 노트북의 커널로 등록하기 위한 패키지 설치
pip install ipykernel

# 가상환경을 'ML Environment'라는 이름의 커널로 등록
python -m ipykernel install --user --name ml_env --display-name "ML Environment"
```

Step 3: VS Code 설정 및 실행

1. **VS Code 설치:** code.visualstudio.com에서 설치합니다.
2. **확장 프로그램 설치:** 왼쪽 Extensions(Ctrl+Shift+X)에서 `Python` 과 `Jupyter` 를 검색하여 설치합니다.
3. **파일 생성:** `ml_study` 폴더를 열고, `test.ipynb` 파일을 생성합니다.
4. **커널 선택:** 오른쪽 상단의 `Select Kernel` $\rightarrow$ `Python Environments` $\rightarrow$ 우리가 등록한 `ML Environment` (또는 `ml_env`)를 선택합니다.

[노트북 셀에 입력할 코드]

```
import sklearn
import pandas as pd
import numpy as np

print(f"scikit-learn 버전: {sklearn.__version__}")
print(f"Pandas 버전: {pd.__version__}")
print(f"NumPy 버전: {np.__version__}")
print(" 머신러닝 환경 구축 완료!")
```

실행 결과 해설

코드를 실행하면 다음과 유사한 결과가 출력됩니다.

```
scikit-learn 버전: 1.3.0
Pandas 버전: 2.1.0
NumPy 버전: 1.26.0
머신러닝 환경 구축 완료!
```

- **버전 숫자:** 설치된 라이브러리의 버전입니다. 머신러닝 모델은 버전에 따라 결과가 미세하게 달라질 수 있어, 협업 시 이 정보를 공유하는 것이 중요합니다.
- **성공 메시지:** `ImportError` 없이 메시지가 출력되었다면, 파이썬이 가상환경 내의 커널을 통해 라이브러리들을 정확히 찾아내어 로드했다는 뜻입니다.

실무 활용 사례

실무 데이터 과학자들은 주피터 노트북과 VS Code를 다음과 같이 조합하여 사용합니다. 1. **데이터 탐색(EDA):** VS Code 내의 `.ipynb` 파일에서 데이터를 불러와 결측치를 확인하고 시각화하며 가설을 세웁니다. 2. **실험 기록:** 마크다운 셀을 활용해 "왜 이 모델을 선택했는지", "파라미터를 바꿨을 때 성능이 어떻게 변했는지"를 기록하여 나중에 다시 봐도 이해할 수 있는 리포트로 만듭니다. 3. **모듈화:** 실험이 끝나고 성능이 검증된 코드는 `.py` 파일로 옮겨 실제 서비스에 적용할 수 있는 형태로 정제합니다. 4. **협업:** 완성된 노트북 파일을 팀원에게 공유하여, 팀원이 동일한 가상환경(커널) 설정만 한다면 내 실험 결과를 100% 재현할 수 있게 합니다.

자주 하는 실수

파이썬 설치 시 'Add Python to PATH'를 체크하지 않은 경우 - 증상: 터미널에 `python` 이라고 입력하면 "명령어를 찾을 수 없습니다"라는 오류가 발생합니다. - 해결: 파이썬 설치 프로그램을 다시 실행하여 'Modify'를 선택하고 PATH 추가 옵션을 체크하거나, 삭제 후 재설치하세요.

가상환경을 활성화하지 않고 `pip install` 을 하는 경우 - 증상: 분명히 설치했는데 주피터 노트북이나 VS Code에서 `ModuleNotFoundError` 가 발생합니다. - 원인: 가상환경(`mL_env`)이 아닌 시스템 전체 파이썬 영역에 설치했기 때문입니다. - 해결: 터미널 프롬프트 앞에 `(mL_env)` 표시가 있는지 확인하고, 없다면 `activate` 명령어를 먼저 실행하세요.

가상환경은 만들었지만 커널 등록(`ipykernel`)을 하지 않은 경우 - 증상: VS Code 커널 선택 목록에 내가 만든 가상환경이 나타나지 않거나, 선택해도 라이브러리를 인식하지 못합니다. - 원인: 가상환경(폴더)만 만들었을 뿐, 주피터 노트북이 사용할 수 있는 '엔진(커널)'으로 등록하지 않았기 때문입니다. - 해결: `pip install ipykernel` 설치 후 `python -m ipykernel install ...` 명령어를 통해 커널을 명시적으로 등록하세요.

핵심 요약

- **머신러닝 환경**은 결과물을 한 번에 내놓는 '제작 환경'이 아니라, 조금씩 수정하며 확인하는 '**대화형 실험 환경**'이다. 특히 메모리에 데이터를 유지하는 '상태 유지' 기능이 핵심이다.
- **가상환경**은 프로젝트별로 라이브러리 버전 충돌을 막기 위한 **독립된 공간(물리적 폴더)**이며, 그 안에서 코드를 실제로 계산해 결과를 돌려주는 백엔드 프로세스를 **커널**이라고 한다.
- **주피터 노트북**은 [가설 $\rightarrow$ 코드 $\rightarrow$ 결과 $\rightarrow$ 해석]의 흐름을 담는 **분석 리포트** 도구이며, **VS Code**는 이를 포함해 전문적인 개발 기능까지 제공하는 **통합 환경**이다.

확인 문제

1. 일반적인 스크립트 방식의 실행과 주피터 노트북과 같은 대화형 방식의 가장 큰 차이점은 무엇인가요?
2. 머신러닝에서 대용량 데이터를 다룰 때 대화형 환경의 '상태 유지(State)' 기능이 왜 중요한가요?
3. 파이썬 설치 시 'Add Python to PATH' 옵션을 선택하지 않으면 어떤 문제가 발생하나요?
4. 가상환경과 커널(Kernel)의 관계를 '방'과 '엔진'의 비유를 들어 설명하세요.
5. 가상환경을 생성한 후, 주피터 노트북에서 이 환경을 인식하게 하려면 어떤 패키지를 설치하고 어떤 과정을 거쳐야 하나요?

정답 및 해설

1. **정답:** 스크립트 방식은 전체 코드를 한 번에 실행하여 결과를 확인하지만, 대화형 방식은 코드를 부분적으로 실행하고 그 결과를 즉시 확인하며 다음 단계를 진행할 수 있다는 점입니다.
2. **정답:** 한 번 로드한 대용량 데이터를 메모리에 유지할 수 있어, 코드를 수정할 때마다 데이터를 다시 읽어오는 시간을 낭비하지 않고 즉시 실험을 반복할 수 있기 때문입니다.
3. **정답:** 운영체제가 파이썬 실행 파일의 경로를 알지 못하므로, 터미널이나 명령 프롬프트에서 `python` 또는 `pip` 명령어를 사용할 수 없게 됩니다.
4. **정답:** 가상환경이 라이브러리들이 격리되어 저장된 '독립된 방'이라면, 커널은 그 방 안에서 실제로 파이썬 코드를 계산하고 결과를 돌려주는 '엔진(백엔드 프로세스)'과 같습니다.
5. **정답:** `ipykernel` 패키지를 설치한 후, `python -m ipykernel install` 명령어를 통해 해당 가상환경을 주피터 커널 목록에 등록해야 합니다.

미니 프로젝트: 나만의 분석 환경 리포트 만들기

목표: VS Code와 주피터 노트북의 기능을 활용해 간단한 환경 설정 리포트를 작성해 보세요.

1. VS Code에서 `env_setup_report.ipynb` 파일을 만듭니다.
2. **마크다운 셀**을 사용하여 `# 나의 머신러닝 실험실 설정`이라는 제목을 적고, 본인의 학습 목표를 간단히 적으세요.
3. **코드 셀**을 만들어 현재 설치된 파이썬 버전과 주요 라이브러리(`numpy`, `pandas`, `sklearn`)의 버전을 출력하는 코드를 작성하세요. (이때 반드시 `ML Environment` 커널을 선택했는지 확인하세요.)
4. **마크다운 셀**을 추가하여, 앞으로 이 환경에서 어떤 데이터를 분석해보고 싶은지 메모하세요.
5. 파일의 내용을 확인하고 저장하세요.

3. NumPy와 Pandas

4. 데이터 시각화

이번 장에서 배우는 내용

데이터 분석의 첫걸음은 숫자의 나열을 '의미 있는 형태'로 바꾸는 것입니다. 이번 장에서는 파이썬의 대표적인 시각화 라이브러리인 Matplotlib과 Seaborn을 사용하여 데이터 속에 숨겨진 패턴을 찾아내는 방법을 배웁니다.

- 데이터 시각화가 머신러닝 모델링에서 왜 중요한지 이해합니다.
 - Matplotlib을 이용해 그래프의 기본 뼈대를 잡는 방법을 익힙니다.
 - Seaborn을 이용해 통계적인 데이터 분포와 상관관계를 효율적으로 분석합니다.
 - 데이터의 성격에 맞는 최적의 그래프 선택 기준을 학습합니다.
 - 실제 데이터셋을 활용해 탐색적 데이터 분석(EDA)의 전체 흐름을 경험합니다.
-

실생활 비유: 엑셀 시트와 사진의 차이

여러분에게 1,000명의 키와 몸무게가 적힌 엑셀 시트가 있다고 가정해 봅시다. 이 표를 아무리 뚫어지게 쳐다봐도 "키가 커질수록 몸무게도 늘어나는 경향이 있구나"라는 것을 단번에 깨닫기는 어렵습니다. 한 줄 한 줄 읽으며 머릿속으로 계산해야 하기 때문입니다.

하지만 이 데이터를 **산점도(Scatter Plot)**라는 그래프로 그리는 순간, 상황은 완전히 달라집니다. 수천 개의 점이 오른쪽 위를 향해 쏠려 있는 '모양'을 보는 순간, 우리는 단 1초 만에 두 변수 사이의 관계를 파악할 수 있습니다.

데이터 시각화는 수천 개의 숫자라는 '텍스트'를 하나의 '사진'으로 바꾸는 과정입니다. 사진을 보면 어디에 이상한 점이 있는지, 전체적인 분위기가 어떤지 즉각적으로 알 수 있듯이, 시각화는 데이터의 '얼굴'을 확인하는 작업입니다.

데이터 시각화의 필요성: 왜 그래프를 그려야 하는가?

우리는 이전 장에서 Pandas의 `.head()` 로 상위 데이터를 확인하고, `.describe()` 로 평균과 표준편차 같은 통계량을 확인했습니다. 하지만 숫자만으로는 알 수 없는 정보들이 많습니다.

예를 들어, 평균값이 50인 데이터셋이 두 개 있다고 합시다. - **데이터 A:** 모든 값이 45~55 사이에 밀집해 있음. - **데이터 B:** 값들이 0과 100에 극단적으로 나뉘어 있음.

두 데이터의 평균은 모두 50이지만, 성격은 완전히 다릅니다. 만약 이 데이터를 그대로 머신러닝 모델에 넣는다면 모델은 매우 혼란스러워할 것입니다. 이처럼 **데이터의 분포, 이상치(Outlier), 변수 간의**

관계는 오직 시각화를 통해서만 직관적으로 파악할 수 있습니다. 이것이 바로 머신러닝 모델을 설계하기 전 반드시 거쳐야 하는 **탐색적 데이터 분석(EDA, Exploratory Data Analysis)**의 핵심입니다.

Matplotlib: 데이터 시각화의 뼈대 만들기

1. 개념 설명: 도화지와 그래프

Matplotlib은 파이썬 시각화의 가장 기초가 되는 라이브러리입니다. 모든 시각화 도구의 '뼈대'라고 생각하면 됩니다. Matplotlib의 구조를 이해하려면 **Figure**와 **Axes**의 관계를 명확히 알아야 합니다.

- **Figure (도화지):** 그래프가 그려질 전체 영역입니다. 커다란 종이 한 장이라고 생각하세요. 하나의 Figure 안에 여러 개의 그래프를 배치할 수 있습니다.
- **Axes (그래프 영역):** 도화지 위에 그려지는 '개별 그래프'입니다. 좌표축(x축, y축)이 있고 실제 데이터가 그려지는 공간입니다.

우리가 흔히 사용하는 `plt.subplots()` 함수는 "도화지(Figure)를 준비하고, 그 안에 그래프 영역(Axes)을 만들어라"라는 명령입니다. 이 함수는 `fig` 와 `ax` 라는 두 개의 객체를 동시에 반환하며, 우리는 `ax` 라는 개별 그래프 영역에 구체적인 그림을 그리게 됩니다.

2. 기술 설명 및 실습

가장 기본이 되는 선 그래프(`plot`)와 산점도(`scatter`)를 그려보겠습니다. 특히 한국어 환경에서는 폰트 깨짐 현상이 발생하므로, 한글 설정 코드를 반드시 포함해야 합니다.

```
import matplotlib.pyplot as plt
import numpy as np

# 한글 폰트 설정 (Windows 기준, Mac은 'AppleGothic')
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

# 1. 데이터 준비
x = np.arange(0, 10, 1) # 0부터 9까지
y = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81] # x의 제곱값

# 2. Figure(도화지)와 Axes(그래프 영역) 생성
fig, ax = plt.subplots(figsize=(8, 5))

# 3. Axes(ax)라는 그래프 영역에 그림 그리기
ax.plot(x, y, color='blue', marker='o', label='제곱 값') # 선 그래프
ax.scatter(x, y, color='red', s=30) # 산점도 (점만 찍기)

# 4. 세부 설정
ax.set_title('x와 y의 관계 시각화', fontsize=15)
ax.set_xlabel('입력 값 (x)')
ax.set_ylabel('결과 값 (y)')
ax.legend()
ax.grid(True)

plt.show()
```

[실행 결과 해설] - 파란색 선과 빨간색 점이 함께 표시됩니다. - `plot` 은 데이터 포인트들을 선으로 연결하여 '흐름'을 보여주고, `scatter` 는 각 데이터의 '위치'를 보여줍니다. - 결과 그래프를 통해 $y = x^2$ 라는 곡선 패턴이 명확하게 드러나는 것을 확인할 수 있습니다.

Seaborn: 더 쉽고 아름답게 데이터 분석하기

1. 개념 설명: Matplotlib의 업그레이드 버전

Matplotlib이 세밀한 조정이 가능한 '수동 도구'라면, Seaborn은 복잡한 설정을 미리 최적화해 둔 '자동 도구'입니다. 특히 Pandas의 DataFrame과 호환성이 매우 높으며, 통계 분석에 특화된 그래프를 단 한 줄의 코드로 그려낼 수 있습니다.

2. 주요 그래프 실습 및 해석

입문자가 반드시 알아야 할 Seaborn의 핵심 그래프들을 하나씩 살펴보겠습니다.

① 히스토그램 (`histplot`): 데이터의 분포 확인

데이터가 어디에 많이 몰려 있는지, 어떤 모양으로 퍼져 있는지를 확인합니다.

```
import seaborn as sns
import matplotlib.pyplot as plt

iris = sns.load_dataset('iris')

plt.figure(figsize=(8, 5))
sns.histplot(data=iris, x='sepal_length', kde=True, color='skyblue')
plt.title('꽃받침 길이 분포')
plt.show()
```

- **해석:** `kde=True` 옵션으로 그려진 부드러운 곡선은 데이터의 밀도를 나타냅니다. 이를 통해 데이터가 정규분포 형태인지, 혹은 한쪽으로 치우쳤는지(Skewness) 확인할 수 있습니다.

② 박스플롯 (`boxplot`): 이상치 확인

데이터의 통계적 요약(최솟값, 1사분위수, 중앙값, 3사분위수, 최댓값)을 한눈에 보여줍니다.

```
plt.figure(figsize=(8, 5))
sns.boxplot(data=iris, x='species', y='sepal_length', palette='Set2')
plt.title('종별 꽃받침 길이 비교')
plt.show()
```

- **해석:** 박스 위아래로 뻗은 선(Whiskers) 밖의 점들이 **이상치(Outlier)**입니다. 보통 **IQR(박스 길이)의 1.5배**를 벗어난 값을 이상치로 판단하며, 이는 모델 학습 전 제거하거나 수정해야 할 후보가 됩니다.

③ 바플롯 (barplot): 그룹 간 평균 비교

범주형 변수에 따른 수치형 변수의 **평균값**을 비교할 때 사용합니다.

```
plt.figure(figsize=(8, 5))
sns.barplot(data=iris, x='species', y='sepal_length', palette='Set2')
plt.title('종별 꽃받침 길이 평균')
plt.show()
```

- **해석:** boxplot 이 데이터의 전체적인 '분포'와 '이상치'를 보여준다면, barplot 은 오직 '평균'이라는 대표값에 집중합니다. 막대 상단의 검은 선은 신뢰구간을 나타내며, 이를 통해 평균값의 불확실성을 함께 파악할 수 있습니다.

④ 산점도 (scatterplot): 두 변수 간의 상관관계 확인

두 수치형 변수가 서로 어떤 관계를 맺고 있는지 점으로 표시합니다.

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=iris, x='sepal_length', y='sepal_width', hue='species')
plt.title('꽃받침 길이 vs 너비')
plt.show()
```

- **해석:** hue 옵션으로 색상을 구분하면, 특정 종이 특정 영역에 뭉쳐 있는 '군집' 패턴을 볼 수 있습니다. 이는 어떤 변수가 클래스를 구분하는 데 유용한지 판단하는 근거가 됩니다.

⑤ 히트맵 (heatmap): 전체 변수 간 상관관계 시각화

여러 변수 간의 상관계수를 색상으로 표현하여 한눈에 파악합니다.

```
plt.figure(figsize=(8, 6))
corr = iris.drop(columns=['species']).corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('변수 간 상관관계')
plt.show()
```

- **해석:** 1에 가까울수록 강한 양의 상관관계, -1에 가까울수록 강한 음의 상관관계를 의미합니다. 0에 가까운 변수는 타겟 변수와 관계가 적으므로 제거 대상이 될 수 있습니다.

데이터의 성격에 맞는 그래프 선택법

분석 목적과 데이터의 타입에 따라 적절한 그래프를 선택하는 것이 중요합니다.

분석 목적	데이터 조합	추천 그래프	핵심 포인트
단일 변수의 분포	수치형 $\rightarrow$ 1개	histplot , kdeplot	데이터의 쏠림, 정규성 확인
그룹 간 비교(분포)	범주형 $\rightarrow$ 수치형	boxplot	중앙값 및 이상치 확인
그룹 간 비교(평균)	범주형 $\rightarrow$ 수치형	barplot	그룹 간 평균 차이 확인
두 변수의 관계	수치형 $\rightarrow$ 수치형	scatterplot , regplot	양/음의 상관관계, 선형성 확인
전체 변수 간 관계	수치형 $\rightarrow$ 수치형 (다수)	heatmap , pairplot	변수 간 얽힘, 다중공선성 확인

실무 활용 사례: 데이터 탐색적 분석(EDA)의 흐름

실무에서는 다음과 같은 유기적인 흐름으로 시각화를 진행합니다. 타이타닉 생존자 데이터셋을 예로 들어 보겠습니다.

```
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터 로드
df = sns.load_dataset('titanic')

# 1. 결측치 확인 (Missing Value Heatmap)
plt.figure(figsize=(10, 6))
sns.heatmap(df.isnull(), cbar=False, yticklabels=False, cmap='viridis')
plt.title('결측치 분포 확인')
plt.show()
```

[EDA 단계별 상세 흐름] 1. 결측치 확인: 위 코드의 히트맵에서 노란색으로 표시되는 부분이 결측치 (True)입니다. 특정 컬럼(예: age 나 deck)에 노란색 줄 형태의 빈 공간이 길게 나타난다면, 해당 변수에 데이터 누락이 심각하다는 것을 직관적으로 알 수 있습니다. 2. 분포 확인: sns.histplot(df['age']) 등을 통해 나이 분포를 확인합니다. 데이터가 한쪽으로 치우쳐 있다면 로그 변환이나 스케일링이 필요함을 판단합니다. 3. 상관관계 분석: sns.heatmap(df.corr()) 를 통해 생존 여부 (survived)와 가장 관계가 깊은 변수(예: 객실 등급 pclass)를 찾아냅니다. 4. 이상치 탐색: sns.boxplot(x='survived', y='fare', data=df) 를 통해 운임 (fare) 데이터에서 상식을 벗어난 극단적인 고액 결제자가 있는지 확인하고 처리합니다.

이 과정을 거치지 않고 바로 모델을 학습시키면, "쓰레기를 넣으면 쓰레기가 나온다(Garbage In, Garbage Out)"는 원칙에 따라 성능이 낮은 모델이 만들어지게 됩니다.

시각화 과정에서 자주 하는 실수

1. 축 범위(Scale) 조작으로 인한 해석의 차이

Y축의 시작점을 0이 아닌 특정 값으로 설정하면, 아주 작은 차이가 매우 커 보이는 착시 현상이 발생할 수 있습니다. - **주의점:** 분석가는 자신의 의도가 독자에게 오해를 불러일으키지 않도록 축 범위를 명시적으로 표시하거나, 데이터의 전체 맥락을 고려하여 설정해야 합니다.

2. 과도한 색상 사용

너무 많은 색상을 사용하면 오히려 시선이 분산되어 핵심 패턴을 놓치게 됩니다. `hue` 옵션은 꼭 필요한 구분 기준에만 사용하세요.

3. 데이터 타입 무시

범주형 데이터(예: 도시 이름)를 숫자로 변환하지 않고 그대로 산점도에 넣으려 하면 에러가 발생합니다. 시각화 전 반드시 `df.info()` 로 데이터 타입을 확인하세요.

핵심 요약

- 시각화는 숫자로 된 데이터를 '패턴'과 '모양'으로 바꾸어 직관적으로 이해하는 과정이다.
- **Matplotlib**은 기초 뼈대(Figure: 도화지, Axes: 그래프 영역)를 잡는 도구이며, **Seaborn**은 통계 분석에 최적화된 고수준 도구이다.
- **분포**는 히스토그램, **이상치**는 박스플롯, **평균 비교**는 바플롯, **관계**는 산점도와 히트맵으로 확인한다.
- 시각화 $\rightarrow$ 데이터 정제 $\rightarrow$ 모델링 순으로 이어지는 **EDA** 과정이 머신러닝 성능의 핵심이다.

확인 문제

1. Matplotlib에서 전체 도화지와 실제 그래프 영역을 지칭하는 용어는 각각 무엇인가요?
2. 데이터의 이상치(Outlier)를 가장 직관적으로 확인할 수 있는 Seaborn 그래프는 무엇이며, 일반적으로 어떤 기준(IQR 관련)으로 판단하나요?
3. 두 수치형 변수 간의 상관관계를 수치와 색상으로 한눈에 보여주는 그래프는 무엇인가요?

[정답 및 해설] 1. **Figure**와 **Axes**입니다. Figure는 전체 캔버스, Axes는 그 안의 개별 그래프를 의미합니다. 2. **boxplot** (박스플롯)입니다. 박스 길이(IQR)의 1.5배를 벗어난 지점의 점들을 이상치로 판단합니다. 3. **heatmap** (히트맵)입니다. 상관관계수 행렬을 색상으로 표현하여 관계의 강도를 보여줍니다.

미니 프로젝트: 타이타닉 생존자 데이터 분석하기

목표: 타이타닉 데이터셋을 활용해 어떤 조건의 사람들이 더 많이 생존했는지 시각적으로 분석해 보세요.

- **데이터 로드:** `sns.load_dataset('titanic')`
- **미션:** 1. 성별(`sex`)에 따른 생존율(`survived`)을 `barplot`으로 그려보세요. 2. 객실 등급(`pclass`)과 생존율의 관계를 `boxplot`이나 `countplot`으로 확인해 보세요. 3. 나이(`age`)의 분포를 `histplot`으로 그려보고, 생존 여부에 따라 색상을 다르게 표시해 보세요.
- **결론 도출:** 시각화 결과를 바탕으로 "어떤 특성을 가진 사람이 생존 확률이 높았는가?"에 대한 가설을 세워보세요.

5. 데이터 전처리

이번 장에서 배우는 내용

머신러닝 모델의 성능을 결정짓는 것은 화려한 알고리즘보다 '**데이터의 품질**'인 경우가 훨씬 많습니다. 아무리 뛰어난 요리사(모델)라도 재료(데이터)가 썩어 있거나 손질되지 않았다면 훌륭한 요리를 만들어낼 수 없습니다.

현실 세계의 데이터는 매우 지저분합니다. 값이 비어 있기도 하고(결측치), 말도 안 되게 크거나 작은 값이 섞여 있으며(이상치), 컴퓨터가 이해하지 못하는 문자열 형태로 되어 있기도 합니다. 이러한 데이터를 모델이 학습할 수 있는 최적의 형태로 가공하는 과정을 '**데이터 전처리(Data Preprocessing)**'라고 합니다.

이번 장에서는 머신러닝의 필수 관문인 데이터 전처리의 핵심 기법들을 배웁니다.

- **결측치 처리**: 비어 있는 데이터를 어떻게 채우거나 제거할 것인가?
- **이상치 처리**: 데이터의 흐름을 방해하는 튀는 값을 어떻게 처리할 것인가?
- **피쳐 스케일링**: 서로 다른 단위의 숫자 데이터를 동일한 기준으로 맞추는 정규화와 표준화.
- **범주형 데이터 인코딩**: 문자열 데이터를 컴퓨터가 계산할 수 있는 숫자로 바꾸는 방법.
- **전처리 파이프라인**: 실무에서 효율적으로 전처리를 적용하는 흐름.

실생활 비유: 요리 전 '재료 손질'

여러분이 맛있는 찌개를 끓이려고 시장에서 재료를 사 왔다고 상상해 보세요.

시장에서 사 온 재료를 그대로 냄비에 넣으시겠습니까? 아마 아닐 것입니다. * **결측치 처리**: 썩은 부분은 도려내고, 부족한 재료는 다른 것으로 대체합니다. * **이상치 처리**: 너무 거대한 무 조각이 있다면 적당한 크기로 자르거나, 도저히 먹을 수 없는 이물질은 버립니다. * **피쳐 스케일링**: 소금 한 숟가락과 물 한 컵은 양의 단위가 다릅니다. 이를 '간'이라는 하나의 기준에 맞춰 조절해야 맛이 균형 잡힙니다. * **인코딩**: '매운맛', '순한맛'이라는 추상적인 맛의 종류를 '고춧가루 3스푼', '고춧가루 1스푼'이라는 구체적인 양(숫자)으로 변환하여 넣습니다.

이처럼 재료를 깨끗이 씻고, 다듬고, 양을 맞추는 '**재료 손질**' 과정이 끝나야 비로소 요리(모델 학습)를 시작할 수 있습니다. 데이터 전처리는 바로 머신러닝을 위한 재료 손질 과정입니다.

개념 설명: 왜 전처리가 필요한가?

1. Garbage In, Garbage Out (GIGO)

"쓰레기가 들어가면 쓰레기가 나온다"는 뜻입니다. 모델이 아무리 복잡하고 정교해도, 입력 데이터에 오류가 많거나 편향되어 있다면 모델은 잘못된 패턴을 학습하게 됩니다.

2. 모델의 수학적 특성

머신러닝 모델은 내부적으로 거대한 수학 계산기입니다. * **숫자만 이해함**: "서울", "부산" 같은 문자열은 계산할 수 없습니다. $\rightarrow$ **인코딩 필요** * **단위에 민감함**: '나이(0~100)'와 '연봉(0~수억)'이 함께 있으면, 모델은 숫자가 큰 연봉이 훨씬 더 중요한 정보라고 착각합니다. $\rightarrow$ **스케일링 필요** * **빈 값에 취약함**: 수학 식에 NaN (Not a Number)이 들어가면 결과는 모두 NaN 이 됩니다. $\rightarrow$ **결측치 처리 필요**

기술 설명: 전처리의 4대 핵심 기법

1. 결측치(Missing Value) 처리

데이터가 누락된 상태를 말합니다. * **제거(Deletion)**: 결측치가 너무 많은 행이나 열을 삭제합니다. (데이터 손실 위험) * **대체(Imputation)**: 평균값, 중앙값, 최빈값 등으로 빈칸을 채웁니다.

2. 이상치(Outlier) 처리

대부분의 데이터 범위에서 크게 벗어난 값입니다. * **검출**: IQR(Interquartile Range) 방식이나 Z-Score를 사용하여 판단합니다. * **처리**: 이상치를 제거하거나, 상한/하한값으로 강제 조정(Clipping)합니다.

3. 피쳐 스케일링(Feature Scaling)

데이터의 범위를 일정하게 맞추는 작업입니다. * **정규화(Normalization/Min-Max Scaling)**: 모든 값을 0과 1 사이로 변환합니다. (데이터의 분포가 균일할 때 유리) * **표준화(Standardization)**: 평균을 0, 표준편차를 1로 변환합니다. (이상치가 포함되어 있을 때 더 안정적)

4. 범주형 인코딩(Categorical Encoding)

문자 데이터를 숫자 데이터로 변환합니다. * **레이블 인코딩(Label Encoding)**: 각 항목에 고유한 숫자(0, 1, 2...)를 부여합니다. (순서 의미가 있을 때 사용) * **원-핫 인코딩(One-Hot Encoding)**: 항목별로 새로운 열을 만들고 해당하면 1, 아니면 0을 부여합니다. (순서 의미가 없을 때 사용)

파이썬 코드 실습

pandas 와 scikit-learn 을 사용하여 지저분한 데이터를 깨끗하게 만들어 보겠습니다.

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

# 1. 지저분한 샘플 데이터 생성
data = {
    'Age': [25, 30, np.nan, 45, 200], # 결측치(nan)와 이상치(200) 포함
    'Salary': [5000, 6000, 5500, np.nan, 6200], # 결측치 포함
    'City': ['Seoul', 'Busan', 'Seoul', 'Incheon', 'Busan'], # 범주형 데이터
    'Target': [0, 1, 0, 1, 0]
}
df = pd.DataFrame(data)
print("--- 원본 데이터 ---\n", df, "\n")

# 2. 결측치 처리 (평균값으로 채우기)
imputer = SimpleImputer(strategy='mean')
df[['Age', 'Salary']] = imputer.fit_transform(df[['Age', 'Salary']])

# 3. 이상치 처리 (Age가 100세 이상이면 100세로 조정 - Clipping)
df['Age'] = df['Age'].clip(upper=100)

# 4. 피쳐 스케일링 (표준화 적용)
scaler = StandardScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])

# 5. 범주형 인코딩 (원-핫 인코딩)
# pandas의 get_dummies를 사용하면 간편하게 구현 가능합니다.
df = pd.get_dummies(df, columns=['City'])

print("--- 전처리 후 데이터 ---\n", df)

```

실행 결과 해설

- 결측치 처리:** Age의 NaN은 나머지 값들의 평균으로 채워졌습니다. 이제 수학적 계산이 가능해졌습니다.
- 이상치 처리:** Age의 200세라는 비현실적인 값은 `.clip(upper=100)`을 통해 100세로 조정되었습니다. 모델이 200세라는 값에 휘둘리는 것을 방지합니다.
- 표준화:** Age와 Salary는 원래 단위가 완전히 달랐지만, `StandardScaler`를 거치며 평균 0, 표준편차 1인 비슷한 범위의 숫자로 변환되었습니다.
- 원-핫 인코딩:** City 열이 사라지고 `City_Seoul`, `City_Busan`, `City_Incheon`이라는 3개의 열이 생겼습니다. 각 행은 자신이 해당하는 도시에만 1을 가지고 나머지는 0을 가집니다.

실무 활용 사례: 고객 이탈 예측 시스템

은행에서 고객이 서비스를 해지할지 예측하는 모델을 만든다고 가정해 봅시다.

- 문제:** 고객 데이터에는 '직업'이라는 문자열 필드와 '잔고'라는 숫자 필드가 있습니다. 일부 고객은 '소득' 정보를 입력하지 않았습니다.
- 적용:**
 - 소득의 빈칸은 전체 고객의 중앙값으로 채워 데이터 손실을 막습니다.

2. **잔고** 데이터 중 억만장자 수준의 극단적 이상치는 로그 변환(Log Transform)을 통해 범위를 좁힙니다.
 3. **직업** 필드는 원-핫 인코딩을 통해 숫자로 변환하여 모델에 입력합니다.
- **결과:** 전처리를 거친 데이터로 학습시킨 모델은 전처리 전보다 예측 정확도가 평균 15~20% 향상되는 결과를 보입니다.

자주 하는 실수

실수: 전체 데이터셋을 한 번에 스케일링하기 * **원인:** 학습 데이터(Train)와 테스트 데이터(Test)를 나누기 전에 전체 데이터에 `fit_transform()` 을 적용하는 경우입니다. * **결과:** 데이터 누수(Data Leakage)가 발생합니다. 테스트 데이터의 정보(평균, 표준편차)가 학습 과정에 미리 반영되어, 실제 서비스 시보다 성능이 과하게 높게 나오는 '낙관적 편향'이 생깁니다. * **해결:** 반드시 데이터를 먼저 나누고, **학습 데이터로만 fit** 을 한 뒤, 그 기준을 테스트 데이터에 **transform** 만 적용하세요.

핵심 요약

- **데이터 전처리**는 모델 학습 전 데이터를 깨끗하고 일관된 형태로 만드는 필수 과정입니다.
- **결측치**는 제거하거나 평균/중앙값 등으로 대체하여 처리합니다.
- **이상치**는 IQR 등의 방법으로 검출하고, 제거하거나 특정 범위로 제한(Clipping)합니다.
- **스케일링**은 정규화(0~1)와 표준화(평균 0, 분산 1)가 있으며, 변수 간 단위 차이를 없애줍니다.
- **인코딩**은 문자열 데이터를 숫자로 바꾸는 과정으로, 레이블 인코딩과 원-핫 인코딩이 대표적입니다.

확인 문제

1. 데이터 전처리 과정에서 '정규화(Normalization)'와 '표준화(Standardization)'의 가장 큰 차이점은 무엇인가요?
2. 범주형 데이터 중 '학력(초졸, 중졸, 고졸, 대졸)'과 같이 순서 의미가 있는 데이터를 인코딩할 때, 원-핫 인코딩보다 더 적절한 방법은 무엇인가요?
3. 데이터 누수(Data Leakage)를 방지하기 위해 스케일링을 적용할 때 주의해야 할 점은 무엇인가요?

정답 및 해설

1. **정답:** 정규화는 데이터를 0과 1 사이의 일정한 범위로 압축하는 것이고, 표준화는 데이터를 평균 0, 표준편차 1인 분포로 변환하는 것입니다. 정규화는 데이터의 분포가 균일할 때 좋고, 표준화는 이상치가 있을 때 더 강건(Robust)합니다.
2. **정답:** 레이블 인코딩(Label Encoding) 또는 오디널 인코딩(Ordinal Encoding)입니다. 순서 정보가 중요하므로 \$초졸=1, 중졸=2, \dots\$ 와 같이 숫자의 크기로 순서를 표현하는 것이 유리합니다.
3. **정답:** 학습 데이터와 테스트 데이터를 먼저 분리한 후, 학습 데이터의 통계량(평균, 표준편차 등)만을 사용하여 스케일러를 학습(fit)시켜야 합니다. 테스트 데이터에는 학습된 스케일러를 적용(transform)만 해야 합니다.

미니 프로젝트: 중고차 가격 예측 전처리 파이프라인 만들기

목표: 중고차 매물 데이터를 분석하여 가격 예측 모델에 넣기 전까지의 모든 전처리 과정을 구현해 보세요.

데이터 조건 (가상): * **Year**: 제조 연도 (정수, 일부 누락) * **Mileage**: 주행 거리 (정수, 일부 누락, 100만km 이상의 이상치 존재) * **Fuel_Type**: 연료 타입 (Gasoline, Diesel, Electric / 문자열) * **Brand**: 브랜드 (Hyundai, Kia, BMW, Mercedes / 문자열) * **Price**: 판매 가격 (타겟 변수)

요구사항: 1. **Year** 와 **Mileage** 의 결측치를 중앙값(Median)으로 채우세요. 2. **Mileage** 에서 상위 1%에 해당하는 값을 이상치로 판단하여 상한값으로 Clipping 하세요. 3. **Fuel_Type** 과 **Brand** 를 원-핫 인코딩으로 변환하세요. 4. **Year** 와 **Mileage** 를 **StandardScaler** 를 사용하여 표준화하세요. 5. 최종적으로 전처리가 완료된 DataFrame의 형태(Shape)와 상위 5개 행을 출력하는 코드를 작성하세요.

6. 지도학습

제시해주신 피드백을 바탕으로, 지도 학습의 핵심 개념인 '레이블(Label)'의 중요성을 강조하고, 학습 프로세스를 구체화하여 내용을 보강하였습니다.

[수정본] 00. 지도 학습 (Supervised Learning)

지도 학습은 정답(Label)이 포함된 데이터를 사용하여 모델을 학습시키는 방법입니다. 마치 선생님이 학생에게 문제와 정답을 함께 알려주며 학습시키는 과정과 유사하여 '지도 학습'이라고도 불립니다.

1. 지도 학습의 핵심: 레이블(Label)

지도 학습의 가장 큰 특징은 학습 데이터가 '특성(Feature)'과 '레이블(Label)'의 쌍으로 이루어져 있다는 점입니다.

- **특성(Feature):** 예측을 위해 사용되는 입력 값 (예: 집의 크기, 위치, 방 개수)
- **레이블(Label):** 우리가 맞추고자 하는 정답 값 (예: 집값)

모델은 수많은 데이터 쌍을 학습하며 "어떤 특성들이 나타났을 때 어떤 레이블이 도출되는가"에 대한 규칙(함수)을 찾아내고, 학습이 완료되면 정답이 없는 새로운 데이터가 들어왔을 때 그 정답을 예측하게 됩니다.

2. 지도 학습의 프로세스 (Workflow)

지도 학습은 단순히 데이터를 넣는 것이 아니라, 다음과 같은 체계적인 과정을 거쳐 수행됩니다.

1. **데이터 수집 및 레이블링:** 문제(특성)와 정답(레이블)이 짝지어진 데이터를 수집합니다.
 2. **데이터 분할 (Split):** 수집한 데이터를 보통 **학습 세트(Training set)**와 **테스트 세트(Test set)**로 나눕니다.
 - **학습 세트:** 모델이 규칙을 찾기 위해 공부하는 데이터입니다.
 - **테스트 세트:** 학습에 사용되지 않은 데이터로, 모델이 얼마나 정확하게 예측하는지 최종 검증하는 용도입니다.
 3. **모델 학습 (Training):** 학습 세트를 통해 특성과 레이블 사이의 관계를 학습하여 최적의 모델을 생성합니다.
 4. **평가 및 튜닝 (Evaluation & Tuning):** 테스트 세트를 통해 예측값과 실제 정답을 비교하여 성능을 평가하고, 필요시 하이퍼파라미터를 조정하여 성능을 높입니다.
 5. **배포 및 예측 (Deployment):** 완성된 모델에 새로운 데이터를 입력하여 결과를 예측합니다.
-

3. 지도 학습의 주요 유형

지도 학습은 해결하려는 문제의 성격(레이블의 형태)에 따라 크게 두 가지로 나뉩니다.

① 회귀 (Regression)

- **목적:** 연속적인 수치(Number)를 예측하는 것
- **특징:** 결과값이 숫자 형태이며, "얼마나(How much)"에 대한 답을 찾습니다.
- **예시:**
 - 과거 데이터를 바탕으로 내일의 기온 예측하기
 - 아파트의 조건(특성)을 보고 매매가(레이블) 예측하기
 - 공부 시간에 따른 시험 점수 예측하기

② 분류 (Classification)

- **목적:** 미리 정의된 범주(Category/Class) 중 하나를 예측하는 것
- **특징:** 결과값이 이산적인 클래스 형태이며, "어떤 것(Which one)"에 대한 답을 찾습니다.
- **예시:**
 - 이메일의 텍스트를 분석해 '스팸'인지 '정상'인지 판별하기
 - 사진 속 동물이 '개'인지 '고양이'인지 구분하기
 - 환자의 증상을 보고 질병의 유무(양성/음성) 판단하기

[요약] 회귀 vs 분류 비교

구분	회귀 (Regression)	분류 (Classification)
출력 값의 형태	연속적인 수치 (Continuous)	이산적인 범주 (Discrete)
핵심 질문	"얼마나(How much)?"	"어떤 것(Which one)?"
결과 예시	25.4도, 5억 원, 85점	스팸/정상, 개/고양이, 합격/불합격
목표	값의 오차를 최소화하는 것	정답 클래스를 정확히 맞추는 것

7. 회귀

이번 장에서 배우는 내용

이번 장에서는 머신러닝의 가장 대표적인 지도학습 기법 중 하나인 '**회귀(Regression)**'에 대해 배웁니다. 단순히 데이터를 분석하는 것을 넘어, 미래의 수치를 예측하는 모델을 만드는 과정을 다룹니다.

- 회귀의 개념과 분류(Classification)와의 차이점
 - 선형 회귀의 원리와 최적의 직선을 찾는 방법
 - Scikit-learn 라이브러리를 이용한 회귀 모델 구현
 - 예측 모델의 정확도를 측정하는 평가 지표(MAE, MSE, R^2)
 - 여러 개의 변수를 사용하는 다중 선형 회귀로의 확장
-

실생활 비유: "공부 시간과 시험 점수의 상관관계"

여러분은 시험 전날, 친구들과 이런 이야기를 나눈 적이 있을 것입니다. "보통 1시간 공부할 때마다 5점 정도 오르는 것 같아. 내가 10시간을 공부하면 대략 50점 정도는 나오겠지?"

여기서 여러분은 무의식중에 '**회귀 분석**'을 하고 있는 것입니다. 과거의 경험(데이터)을 바탕으로 '**공부 시간**'이라는 입력값이 변할 때 '**시험 점수**'라는 결과값이 어떻게 변하는지 그 규칙성을 찾아낸 것이죠.

만약 여러분이 이 규칙을 그래프로 그린다면, 점들이 흩어져 있겠지만 그 중심을 가로지르는 하나의 '**직선**'을 그릴 수 있을 것입니다. 이 직선이 바로 머신러닝이 찾고자 하는 '**최적의 예측 모델**'입니다.

개념 설명: 회귀(Regression)란 무엇인가?

회귀란 쉽게 말해 "**연속적인 수치를 예측하는 것**"입니다.

우리가 앞서 배운 '지도학습'은 크게 두 가지 갈래로 나뉩니다. 하나는 **분류(Classification)**이고, 다른 하나가 바로 **회귀(Regression)**입니다. 이 둘의 차이를 명확히 아는 것이 중요합니다.

- **분류 (Classification):** "이 사진은 고양이인가, 강아지인가?", "이 메일은 스팸인가, 정상인가?"처럼 **범주(Category)**를 맞추는 것입니다. 결과값이 'A 아니면 B'로 딱 떨어집니다.
- **회귀 (Regression):** "내일의 기온은 몇 도일까?", "이 아파트의 적정 가격은 얼마일까?", "다음 달 매출액은 얼마일까?"처럼 **연속적인 숫자(Continuous Value)**를 맞추는 것입니다.

즉, 정답이 '예/아니오'나 '종류'가 아니라 '**얼마나(How much)**'에 해당하는 수치라면 그것은 회귀 문제라고 할 수 있습니다.

기술 설명: 선형 회귀의 원리

1. 최적의 선을 찾는 방정식

선형 회귀(Linear Regression)는 데이터의 분포를 가장 잘 나타내는 하나의 직선을 찾는 것입니다. 중학교 수학 시간에 배운 일차함수 식을 떠올려 보세요.

$$y = ax + b$$

- **x (독립변수):** 예측을 위해 사용하는 입력값 (예: 공부 시간)
- **y (종속변수):** 우리가 예측하려는 결과값 (예: 시험 점수)
- **a (기울기/Weight):** x 가 변할 때 y 가 얼마나 변하는지를 나타내는 가중치
- **b (절편/Bias):** x 가 0일 때의 기본값 (편향)

머신러닝 모델의 학습이란, 수많은 데이터들을 가장 잘 설명할 수 있는 **최적의 a 와 b 값을 찾아내는 과정**을 말합니다.

2. 손실 함수(Loss Function): "얼마나 틀렸는가?"

컴퓨터는 어떻게 '최적의 선'을 찾을까요? 바로 '**오차**'를 계산하기 때문입니다.

- **실제값:** 실제 데이터에 기록된 정답
- **예측값:** 모델이 직선($y = ax + b$)을 통해 계산한 값

이 실제값과 예측값의 차이를 **오차(Error)**라고 하며, 이 오차들의 합이나 평균을 계산하는 식을 **손실 함수(Loss Function)**라고 합니다. 모델은 이 손실 함수의 값이 최소가 되도록(즉, 오차가 가장 적어지도록) a 와 b 를 계속해서 수정합니다.

파이썬 코드 실습: Scikit-learn으로 회귀 모델 만들기

이제 실제로 파이썬의 `Scikit-learn` 라이브러리를 사용하여 집값 예측 모델을 만들어 보겠습니다. 집의 크기(평수)에 따라 가격이 결정된다는 단순한 가정을 세워보겠습니다.

중요: 데이터 모양(Shape) 주의사항 Scikit-learn의 회귀 모델은 입력 데이터인 x 를 **2차원 배열(행렬)** 형태로 기대합니다. - **잘못된 형태:** `[10, 15, 20]` (1차원 리스트) $\rightarrow$ 모델이 "특성이 몇 개인지 모르겠다"며 에러를 냅니다. - **올바른 형태:** `[[10], [15], [20]]` (2차원 배열) $\rightarrow$ "데이터가 3개이고, 각 데이터당 특성이 1개구나"라고 인식합니다. 그래서 코드에서 `.reshape(-1, 1)`을 사용하여 1차원 데이터를 2차원으로 변환해 주는 것입니다.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# 1. 데이터 준비 (X: 집 크기(평수), y: 집값(억 원))
# reshape(-1, 1)을 통해 1차원 배열을 2차원 배열로 변환합니다.
X = np.array([10, 15, 20, 25, 30, 35, 40]).reshape(-1, 1)
y = np.array([2.1, 3.2, 4.1, 4.9, 6.1, 7.2, 8.1])

# 2. 모델 생성
model = LinearRegression()

# 3. 모델 학습 (fit 메서드 사용)
# 데이터 사이의 최적의 직선(기울기와 절편)을 찾는 과정입니다.
model.fit(X, y)

# 4. 새로운 데이터로 예측 (predict 메서드 사용)
# 예측할 때도 X 데이터는 2차원 형태([[값]])여야 합니다.
new_house_size = np.array([[28]])
predicted_price = model.predict(new_house_size)

print(f"28평 아파트의 예측 가격은 {predicted_price[0]:.2f}억 원입니다.")

# 5. 결과 시각화
plt.scatter(X, y, color='blue', label='Actual Data') # 실제 데이터 점
plt.plot(X, model.predict(X), color='red', label='Regression Line') # 학습된 예측 직선
plt.xlabel('House Size (pyeong)')
plt.ylabel('Price (100M KRW)')
plt.legend()
plt.show()
```

실행 결과 해설

- `model.fit(X, y)` : 이 메서드가 실행될 때 머신러닝 모델은 내부적으로 데이터를 훑으며 오차를 최소화하는 최적의 기울기(\$a\$)와 절편(\$b\$)을 계산합니다.
- `model.predict([[28]])` : 학습된 직선 방정식에 $x=28$ 을 대입하여 y 값을 출력합니다.
- **그래프 결과**: 파란색 점들(실제 데이터) 사이를 붉은색 직선이 관통하는 것을 볼 수 있습니다. 이 직선이 바로 모델이 학습한 '데이터의 경향성'입니다.

모델의 성적표: 회귀 모델 평가 지표

모델이 예측값을 내놓았지만, 이 모델이 얼마나 믿을만한지 정량적으로 확인해야 합니다. 이때 사용하는 지표들이 있으며, 각각의 목적이 다릅니다.

1. **MAE (Mean Absolute Error, 평균 절대 오차)**: - 실제값과 예측값의 차이를 절대값으로 변換해 평균을 낸 것입니다. - **특징**: 오차의 크기가 그대로 반영되어 "평균적으로 약 $\text{\$}\{X\}$ 원 정도 틀린다"라고 가장 직관적으로 해석할 수 있습니다.
2. **MSE (Mean Squared Error, 평균 제곱 오차)**: - 오차를 제곱하여 평균을 낸 것입니다. - **특징**: 오차가 클수록 제곱값이 기하급수적으로 커집니다. 따라서 **큰 오차(이상치)**에 대해 매우 민

감하게 반응합니다. 모델이 가끔씩 아주 크게 틀리는 것을 방지하고 싶을 때 유용하게 감시하는 지표입니다.

3. R^2 (R-squared, 결정계수): - 모델이 데이터의 분산을 얼마나 잘 설명하는지를 나타내며, 0에서 1 사이의 값을 가집니다. - 특징: MAE나 MSE는 데이터의 단위(원, 달러 등)에 따라 값이 달라지지만, R^2 는 상대적인 비율이므로 "이 모델이 전체적으로 얼마나 정확한가"를 판단하는 표준 척도가 됩니다.

어떤 지표를 봐야 할까요?

지표	핵심 질문	추천 상황
MAE	"평균적으로 얼마나 틀려?"	일반적인 오차 수준을 직관적으로 알고 싶을 때
MSE	"크게 틀린 데이터가 있어?"	이상치(Outlier)에 민감하게 반응하여 모델을 튜닝할 때
R^2	"모델이 데이터를 얼마나 잘 설명해?"	모델의 전반적인 성능을 다른 모델과 비교하고 싶을 때

더 나은 예측을 향해: 단순 회귀에서 다중 회귀로

지금까지는 '집 크기'라는 하나의 변수만 사용했습니다. 이를 **단순 선형 회귀(Simple Linear Regression)**라고 합니다. 하지만 실제 집값은 크기만으로 결정되지 않습니다.

- 집의 연식 (Age)
- 역세권 여부 (Station Distance)
- 주변 학군 (School District)

이렇게 여러 개의 독립변수($x_1, x_2, x_3 \dots$)를 사용하는 것을 **다중 선형 회귀(Multiple Linear Regression)**라고 합니다.

- **단순 회귀:** $y = ax + b$
- **다중 회귀:** $y = a_1x_1 + a_2x_2 + a_3x_3 + \dots + b$

변수가 많아질수록 모델은 더 복잡한 현실을 반영할 수 있게 되어 예측력이 향상되는 경우가 많습니다. 하지만 여기서 주의해야 할 점이 바로 '**과적합(Overfitting)**'입니다.

과적합(Overfitting)이란? 과적합은 마치 교과서의 연습문제와 정답을 통째로 외워버린 학생과 같습니다. 외운 문제는 완벽하게 맞지만, 조금만 응용된 새로운 문제가 나오면 전혀 풀지 못하는 상태죠.

머신러닝에서도 변수를 너무 많이 넣으면, 모델이 데이터의 일반적인 '경향'을 배우는 것이 아니라 개별 데이터의 '잡음(Noise)'까지 학습해 버립니다. 결과적으로 학습 데이터에 대해서는 완벽한 예측을 하지만, **실제 새로운 데이터(현실)에서는 예측력이 뚝 떨어지는 문제**가 발생합니다.

실무 활용 사례

회귀 모델은 산업 현장에서 매우 광범위하게 사용됩니다. - **이커머스**: 과거 판매 데이터를 기반으로 다음 달 예상 매출액 예측 $\rightarrow$ 재고 물량 조절. - **에너지**: 기온, 습도, 요일 데이터를 기반으로 전력 수요량 예측 $\rightarrow$ 발전소 가동률 조정. - **마케팅**: 광고비 지출액에 따른 예상 신규 고객 유입 수 예측 $\rightarrow$ 광고 예산 최적화.

자주 하는 실수

× **"데이터가 직선 형태가 아닌데 선형 회귀를 사용하는 경우"** - 문제: 데이터가 곡선(예: 지수 함수 형태)으로 분포되어 있는데 억지로 직선을 그으면 오차가 매우 커집니다. - 해결: 데이터를 시각화하여 경향성을 먼저 파악하세요. 곡선 형태라면 '다항 회귀(Polynomial Regression)'나 다른 비선형 모델을 고려해야 합니다.

"단위가 너무 다른 변수들을 그대로 다중 회귀에 넣는 경우" - 문제: 예를 들어 '방 개수(1~5개)'와 '연봉(수천만 원)'을 동시에 넣으면, 숫자가 큰 연봉 변수가 모델에 압도적인 영향을 줍니다. - 해결: 모든 변수의 범위를 일정하게 맞추는 **스케일링(Scaling)** 과정을 거쳐야 합니다.

핵심 요약

- 회귀는 연속적인 수치 값을 예측하는 지도학습 기법이다.
- 선형 회귀는 데이터의 경향성을 가장 잘 나타내는 최적의 직선($y=ax+b$)을 찾는 것이다.
- 손실 함수를 통해 실제값과 예측값의 오차를 줄이는 방향으로 모델을 학습시킨다.
- MAE(직관적 오차), MSE(큰 오차 민감), R^2 (설명력)를 통해 모델의 성능을 평가한다.
- 변수가 하나면 **단순 회귀**, 여러 개면 **다중 회귀**라고 하며, 변수가 너무 많으면 **과적합**이 발생할 수 있다.

확인 문제

1. 다음 중 회귀(Regression) 문제에 해당하는 것은 무엇일까요? - (A) 스팸 메일 분류 - (B) 내일의 최고 기온 예측 - (C) 이미지 속 동물 종류 맞추기
2. 선형 회귀에서 모델이 '학습'한다는 것은 구체적으로 무엇을 찾는 과정인가요?
3. R^2 (결정계수) 값이 0.95라면, 이 모델의 성능은 어떠하다고 판단할 수 있을까요?

정답 및 해설 1. (B): 기온은 연속적인 수치이므로 회귀 문제입니다. (A)와 (C)는 범주를 나누는 분류 문제입니다. 2. 데이터를 가장 잘 설명하는 직선의 기울기(Weight)와 절편(Bias)을 찾는 과정입니다. 3. 매우 우수한 성능입니다. 데이터의 변동성을 95% 정도 설명하고 있다는 뜻입니다.

미니 프로젝트: 우리 동네 카페 매출 예측하기

상황: 당신은 카페 사장님입니다. 일일 방문객 수와 매출액의 관계를 분석하여, 방문객이 100명일 때 매출이 얼마가 될지 예측하는 모델을 만들어 보세요.

제공 데이터: - 방문객 수(X): [20, 35, 50, 65, 80, 95] - 매출액(y, 만원): [15, 28, 42, 55, 70, 88]

미션: 1. 위 데이터를 사용하여 `LinearRegression` 모델을 학습시키세요. (X 데이터의 shape에 주의하세요!) 2. 방문객이 100명일 때의 예상 매출액을 출력하세요. 3. 실제 데이터와 예측 직선을 그래프로 그려 확인하세요.

8. 분류

9. 비지도학습과 군집화

이번 장에서 배우는 내용

지금까지 우리는 정답(Label)이 주어진 데이터를 통해 학습하는 '지도학습'에 대해 배웠습니다. 하지만 현실 세계의 데이터 중에는 정답이 없는 경우가 훨씬 많습니다. 이번 장에서는 정답지 없이 데이터 스스로의 패턴을 찾아내는 **비지도학습(Unsupervised Learning)**의 개념을 이해하고, 그 중 가장 대표적인 알고리즘인 **K-Means 군집화(Clustering)**를 통해 데이터를 의미 있는 그룹으로 묶는 방법을 학습합니다.

정답 없는 학습: 비지도학습의 개념

실생활 비유: 정답지가 있는 문제집 vs 스스로 관찰하는 탐험가

지도학습이 선생님이 옆에서 "이 사진은 고양이야", "이 사진은 강아지야"라고 알려주는 '**정답지가 있는 문제집**'을 푸는 과정이라면, 비지도학습은 아무런 정보 없이 낯선 숲에 들어간 '**탐험가**'와 같습니다. 탐험가는 누가 무엇이라고 알려주지 않아도, "이 나무들은 잎이 넓고 키가 크네", "저 나무들은 잎이 뾰족하고 키가 작네"라며 스스로 공통점과 차이점을 발견해 나무들을 분류하기 시작합니다. 여기서 중요한 점은 탐험가가 이 나무들의 정확한 '이름(정답)'은 모르지만, '**서로 비슷하게 생겼다**'는 패턴을 찾아냈다는 것입니다.

직관적으로 이해하기

비지도학습은 데이터 속에 숨겨진 '**내재적 구조**'를 찾아내는 작업입니다. 정답이 없기 때문에 "이것은 무엇이다"라고 정의 내리는 대신, "이것들은 서로 비슷하다" 혹은 "이것은 다른 것들과 아주 다르다"라는 관계를 찾아내는 것이 핵심입니다.

기술 설명

비지도학습의 주요 목적은 크게 세 가지로 나뉩니다. 1. **군집화(Clustering)**: 유사한 특성을 가진 데이터끼리 그룹으로 묶는 것 (예: 고객 세그먼테이션) 2. **차원 축소(Dimension Reduction)**: 핵심 특징만 남기고 데이터의 복잡도를 줄이는 것 (예: PCA) 3. **연관 규칙 학습(Association Rule Learning)**: 데이터 간의 흥미로운 관계를 찾는 것 (예: "맥주를 사는 사람은 기저귀도 함께 산다")

끼리끼리 묶기: K-Means 군집화의 원리

실생활 비유: 마을의 중심지 정하기

어느 넓은 평원에 흩어져 사는 주민들이 있다고 가정해 봅시다. 이 주민들을 효율적으로 관리하기 위해 K 개의 '마을 회관'을 지으려고 합니다.

1. 처음에는 무작위로 K 개의 장소에 회관을 짓습니다.
2. 주민들은 **자신과 가장 가까운 회관**으로 가서 등록합니다.
3. 회관 관리자는 등록된 주민들의 위치를 모두 고려해, **주민들의 정중앙 지점**으로 회관을 옮깁니다.
4. 회관 위치가 바뀌었으니, 주민들은 다시 **가장 가까운 회관**을 찾아 이동합니다.
5. 더 이상 회관의 위치가 변하지 않을 때까지 이 과정을 반복합니다.

결국, 주민들은 자연스럽게 서로 가까운 사람들끼리 그룹을 이루게 됩니다.

직관적으로 이해하기

K-Means는 "가까운 것들끼리는 같은 그룹일 확률이 높다"라는 단순한 아이디어에서 출발합니다. 여기서 '가깝다'는 기준은 수학적으로 **유클리드 거리(Euclidean Distance)**, 즉 두 점 사이의 직선거리를 의미합니다.

기술 설명: K-Means 작동 프로세스

K-Means 알고리즘은 다음과 같은 반복적인 최적화 과정을 거칩니다.

1. **중심점 초기화**: 데이터 공간에 임의로 K 개의 중심점(Centroid)을 배치합니다.
2. **군집 배정**: 모든 데이터 포인트에 대해 가장 가까운 중심점을 찾아 해당 군집으로 할당합니다.
3. **중심점 업데이트**: 각 군집에 속한 데이터들의 **산술 평균 위치(각 데이터의 좌표값을 모두 더해 데이터 개수로 나눈 지점)**를 계산하여 중심점을 그 위치로 이동시킵니다.
4. **반복**: 중심점의 위치가 더 이상 변하지 않거나, 설정한 최대 반복 횟수에 도달할 때까지 2~3단계를 반복합니다.

파이썬으로 구현하는 K-Means 군집화

이제 Scikit-learn 라이브러리를 사용하여 실제로 데이터를 군집화해 보겠습니다.

파이썬 코드 실습

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

# 1. 가상의 데이터 생성 (4개의 그룹이 섞여 있는 데이터)
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=42)

# 2. 데이터 전처리: 스케일링 (K-Means는 거리 기반이므로 필수!)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. K-Means 모델 생성 및 학습
# n_clusters: 묶고 싶은 그룹의 수(K)
k = 4
kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled) # 학습과 동시에 군집 예측

# 4. 결과 시각화
plt.figure(figsize=(10, 6))

# 군집별 데이터 포인트 출력
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_kmeans, s=50, cmap='viridis')

# 중심점(Centroids) 출력
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='X',
            label='Centroids')

plt.title(f"K-Means Clustering (K={k})")
plt.xlabel("Feature 1 (Scaled)")
plt.ylabel("Feature 2 (Scaled)")
plt.legend()
plt.show()

print(f"최종 중심점 좌표:\n{centers}")
```

실행 결과 해설

- **시각화 결과:** 데이터들이 4개의 색상으로 명확하게 구분되어 나타납니다. 붉은색 'X' 표시가 각 그룹의 중심점(Centroid)입니다.
- **fit_predict():** 모델이 데이터의 패턴을 학습하고, 동시에 각 데이터가 어떤 군집(0, 1, 2, 3)에 속하는지 결과를 반환합니다.
- **cluster_centers_:** 학습이 완료된 후 결정된 최종 중심점의 좌표입니다. 이 좌표는 해당 그룹의 '대표값'이라고 볼 수 있습니다.

최적의 K값 찾기: 엘보우 방법(Elbow Method)

현실에서는 데이터를 몇 개의 그룹(K)으로 나눠야 할지 미리 알 수 없습니다. 이때 사용하는 객관적인 기준이 **엘보우 방법**입니다.

직관적으로 이해하기

그룹의 수(K)를 늘릴수록, 각 데이터와 중심점 사이의 거리는 점점 짧아집니다. 하지만 어느 순간부터는 K 를 늘려도 거리의 감소 폭이 매우 작아지는 지점이 옵니다. 마치 팔을 굽혔을 때 꺾이는 부분인 '**팔꿈치(Elbow)**' 모양과 비슷하다고 하여 엘보우 방법이라 부릅니다.

기술 설명 및 구현

- **SSE (Sum of Squared Errors) / Inertia:** 각 데이터 포인트에서 자신이 속한 군집의 중심점까지의 거리의 제곱 합입니다. Scikit-learn에서는 이를 `inertia_` 라는 속성으로 제공하며, 이 값이 작을수록 군집 내부의 응집도가 높다는 뜻입니다.
- **판단 기준:** K 값을 1부터 차례대로 늘리며 `inertia_` 를 측정합니다. 값이 급격하게 줄어들다가 완만해지는 **그 꺾임 지점(Elbow Point)**이 가장 효율적인 K 값입니다.

엘보우 방법 구현 코드

```
# K값을 1부터 10까지 변화시키며 Inertia 값 측정
inertia_list = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42)
    kmeans.fit(X_scaled)
    inertia_list.append(kmeans.inertia_) # SSE 값 저장

# 엘보우 그래프 시각화
plt.figure(figsize=(8, 5))
plt.plot(K_range, inertia_list, marker='o')
plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia (SSE)")
plt.xticks(K_range)
plt.grid(True)
plt.show()
```

결과 해석: 그래프를 보면 $K=4$ 지점에서 기울기가 급격히 완만해지는 것을 확인할 수 있습니다. 따라서 이 데이터셋의 최적의 K 값은 4라고 판단할 수 있습니다.

주의: K 를 너무 작게 잡으면 서로 다른 특성의 데이터가 한 그룹에 묶이고, 너무 크게 잡으면 의미 없는 세분화(과적합과 유사한 상태)가 일어납니다.

비지도학습의 실무 활용 사례

K-Means는 정답이 없는 상태에서 인사이트를 뽑아내야 하는 비즈니스 환경에서 매우 강력합니다.

1. **고객 세그멘테이션 (Customer Segmentation):** - 구매 금액, 방문 빈도, 체류 시간 등의 데이터를 기반으로 고객을 'VIP', '잠재 충성 고객', '이탈 위험 고객' 등으로 자동 분류하여 맞춤형 마케팅 전략을 세웁니다.
2. **이상치 탐지 (Anomaly Detection):** - 대부분의 데이터가 특정 군집에 모여 있는데, 어떤 군집의 중심점에서도 매우 멀리 떨어져 있는 데이터가 있다면 이를 '이상치(이상 거래, 장비 고장 등)'로 판단할 수 있습니다.
3. **이미지 압축 (Color Quantization):** - 수백만 가지의 색상 값을 K-Means로 묶어 대표 색상 K 개로 단순화함으로써 이미지 용량을 줄이는 기술에 활용됩니다.

초보자가 자주 하는 실수와 해결책

거리 기반 알고리즘에서 스케일링을 생략하는 경우

실수: 키(cm)와 몸무게(kg) 데이터를 그대로 넣고 군집화를 수행합니다. **문제:** 키는 150~190 사이지만, 몸무게는 40~120 사이입니다. 숫자의 범위(Scale)가 다르면, 범위가 큰 특성이 거리 계산에 압도적인 영향을 주어 결과가 왜곡됩니다. **해결:** 반드시 `StandardScaler` 등을 사용하여 모든 특성의 평균을 0, 분산을 1로 맞춰주어야 합니다.

초기 중심점 위치에 따른 결과 변동

실수: 초기 중심점을 완전히 랜덤하게 잡았을 때, 실행할 때마다 결과가 달라지는 현상이 발생합니다. **해결:** Scikit-learn의 `KMeans` 는 기본적으로 `init='k-means++'` 옵션을 사용합니다. 이는 중심점들을 서로 최대한 멀리 떨어뜨려 배치하는 스마트한 초기화 전략으로, 학습 속도를 높이고 결과의 일관성을 보장합니다.

군집의 모양이 구형(Spherical)이 아닐 때

한계: K-Means는 거리를 기준으로 묶기 때문에, 결과적으로 원형(구형) 모양의 군집을 만드는 경향이 있습니다. 만약 데이터가 초승달 모양이나 길쭉한 형태로 분포되어 있다면 K-Means로는 제대로 분류할 수 없습니다. (이 경우 DBSCAN 같은 다른 알고리즘이 필요합니다.)

핵심 요약

- 비지도학습은 정답(Label) 없이 데이터의 내재적 패턴과 구조를 찾는 학습 방식이다.
- K-Means는 중심점과 데이터 사이의 거리를 최소화하여 K 개의 그룹으로 묶는 알고리즘이다.

- **엘보우 방법**은 SSE(Inertia)의 감소 폭이 완만해지는 지점을 찾아 최적의 K 값을 결정하는 방법이다.
- 거리 기반 알고리즘이므로 **특성 스케일링(Feature Scaling)**이 필수적이다.

확인 문제

1. 지도학습과 비지도학습의 가장 결정적인 차이점은 무엇인가요?
2. K-Means 알고리즘에서 중심점을 업데이트하는 기준은 무엇인가요?
3. 왜 K-Means를 사용할 때 `StandardScaler`를 사용하는 것이 권장되나요?

정답 및 해설

1. **정답:** 학습 데이터에 '정답(Label)'이 포함되어 있느냐의 여부입니다. 지도학습은 정답을 통해 예측 모델을 만들고, 비지도학습은 정답 없이 데이터의 구조나 패턴을 발견합니다.
2. **정답:** 해당 군집으로 배정된 모든 데이터 포인트들의 산술 평균 위치(좌표값의 합을 개수로 나눈 지점)로 중심점을 이동시킵니다.
3. **정답:** K-Means는 유클리드 거리를 기반으로 군집을 나누기 때문에, 특성 간의 단위(Scale) 차이가 크면 값이 큰 특성이 결과에 지배적인 영향을 미쳐 왜곡이 발생하기 때문입니다.

미니 프로젝트: 쇼핑몰 고객 분류하기

목표: 고객의 '연간 소득'과 '소비 지수' 데이터를 활용해 고객군을 분류해 보세요.

단계별 가이드라인:

1. **데이터 생성:** `make_blobs(n_samples=500, centers=5, cluster_std=1.0, random_state=42)`를 사용하여 가상의 고객 데이터를 생성하세요. (특성 2개: 소득, 소비 지수)
2. **데이터 정규화:** 거리 기반 알고리즘인 K-Means의 성능을 위해 `StandardScaler`를 적용하여 데이터를 변환하세요.
3. **최적의 K 찾기:** `for` 문을 사용하여 K 를 1부터 10까지 변화시키며 `inertia_` 값을 수집하고, 엘보우 그래프를 그려 최적의 K 를 결정하세요.
4. **군집화 수행:** 결정한 K 값을 `KMeans(n_clusters=k)`에 넣고 `fit_predict()`를 통해 고객들을 그룹화하세요.
5. **결과 분석:** `matplotlib`을 이용해 산점도를 그리고, 각 그룹이 어떤 특성(예: 고소득-저소비 그룹, 저소득-고소비 그룹 등)을 가지는지 분석해 보세요.

10. 모델 평가와 성능 개선

11. 과적합과 하이퍼파라미터 튜닝

이번 장에서 배우는 내용

머신러닝 모델을 만들 때 가장 당혹스러운 순간은 "학습 데이터에서는 성적이 완벽한데, 왜 실제 데이터만 넣으면 엉뚱한 답을 내놓을까?"라는 의문이 들 때입니다. 이는 모델이 데이터의 본질을 이해한 것이 아니라, 정답지를 통째로 외워버렸기 때문에 발생하는 현상입니다.

이번 장을 마치고 나면 여러분은 단순히 정확도가 높은 모델이 아니라, 처음 보는 데이터에서도 유연하게 정답을 찾아내는 '진짜 실력'을 갖춘 모델을 만들 수 있게 됩니다. 구체적으로 다음과 같은 능력을 갖추게 됩니다.

- 과적합과 과소적합을 구분하고, 내 모델이 현재 어떤 상태인지 진단할 수 있습니다.
- 편향-분산 트레이드오프의 원리를 통해 모델의 복잡도를 최적으로 조절할 수 있습니다.
- 하이퍼파라미터 튜닝 전략(Grid/Random Search)을 통해 모델의 잠재 성능을 최대한으로 끌어올릴 수 있습니다.
- L1, L2 규제를 적용해 특정 데이터에 과하게 집착하는 모델의 '욕심'을 제어할 수 있습니다.
- 실무에서 치명적인 실수인 데이터 누수(Data Leakage)를 방지하는 정석적인 파이프라인을 구축할 수 있습니다.

암기왕과 응용력: 과적합(Overfitting)의 정체

실생활 비유: 시험 공부 스타일

두 학생이 수학 시험 공부를 한다고 가정해 봅시다.

- **학생 A (암기왕):** 문제집에 있는 모든 문제와 정답을 통째로 외웠습니다. 똑같은 문제가 나오면 1초 만에 정답을 맞히지만, 숫자 하나만 바뀐 응용 문제가 나오면 완전히 틀려버립니다.
- **학생 B (이해왕):** 공식이 왜 그렇게 도출되는지 원리를 공부했습니다. 문제집 문제는 조금 느리게 풀지만, 처음 보는 응용 문제가 나와도 원리를 적용해 정답을 찾아냅니다.

여기서 **학생 A가 바로 '과적합'된 모델**입니다. 학습 데이터(문제집)에 너무 최적화된 나머지, 새로운 데이터(실전 시험)에 대한 대응력, 즉 **일반화(Generalization)** 능력을 상실한 상태입니다.

개념 설명: 과적합 vs 과소적합

머신러닝 모델이 데이터의 패턴을 학습하는 정도에 따라 두 가지 극단적인 상황이 발생합니다.

1. **과적합 (Overfitting)**: 모델이 학습 데이터의 세세한 소음(Noise)이나 특이값까지 모두 학습해버린 상태입니다. 훈련 데이터에 대한 정확도는 매우 높지만, 새로운 데이터에 대한 정확도는 떨어집니다.
2. **과소적합 (Underfitting)**: 모델이 너무 단순해서 데이터 속에 숨겨진 기본적인 패턴조차 찾아내지 못한 상태입니다. 훈련 데이터와 새로운 데이터 모두에서 성능이 낮게 나타납니다.

기술 설명: 훈련 데이터와 검증 데이터의 분리

내가 만든 모델이 '암기왕'인지 '이해왕'인지 확인하려면 어떻게 해야 할까요? 정답은 **학습에 사용하지 않은 데이터로 시험을 보는 것**입니다.

이를 위해 데이터를 다음과 같이 분리합니다. - **훈련 데이터셋(Train Set)**: 모델이 패턴을 학습하는 데 사용합니다. (문제집) - **검증 데이터셋(Validation Set)**: 학습 중간에 모델의 성능을 확인하고 하이퍼파라미터를 조정하는 데 사용합니다. (모의고사) - **테스트 데이터셋(Test Set)**: 모든 튜닝이 끝난 후 최종 성능을 딱 한 번 평가하는 데 사용합니다. (실전 시험)

실무 활용 사례: 의료 진단 AI

특정 병원의 환자 데이터로 질병 진단 모델을 학습시켰는데, 이 모델이 해당 병원 장비 특유의 노이즈나 환자군의 특성까지 학습해버리는 경우가 있습니다. 이 모델은 그 병원에서는 정확도가 99%지만, 다른 병원 데이터로 테스트하면 정확도가 60%로 떨어집니다. 이때 개발자는 데이터를 더 다양하게 수집하거나 규제 기법을 적용해 '특정 병원만의 특성'이 아닌 '질병의 공통 특성'을 학습하도록 유도합니다.

유연함과 딱딱함의 균형: 편향-분산 트레이드오프

모델의 성능을 결정짓는 두 가지 핵심 요소가 있습니다. 바로 **편향(Bias)**과 **분산(Variance)**입니다.

1. 편향 (Bias) - "너무 딱딱한 생각"

편향은 모델이 데이터의 실제 관계를 너무 단순하게 가정해서 발생하는 오류입니다. - **특징**: "모든 데이터는 직선 형태일 거야!"라고 고집을 부리는 것과 같습니다. - **결과**: 모델이 너무 단순해져서 **과소적합**이 발생합니다.

2. 분산 (Variance) - "너무 유연한 생각"

분산은 모델이 학습 데이터의 작은 변동(Noise)에도 민감하게 반응해서 발생하는 오류입니다. - **특징**: 데이터 하나하나의 움직임에 일일이 반응하여 구불구불한 곡선을 그리는 것과 같습니다. - **결과**: 모델이 너무 복잡해져서 **과적합**이 발생합니다.

트레이드오프 (Trade-off)

편향을 줄이려고 모델을 복잡하게 만들면 분산이 커지고, 분산을 줄이려고 모델을 단순하게 만들면 편향이 커집니다. 머신러닝의 핵심은 **편향과 분산의 합이 최소가 되는 최적의 복잡도**를 찾는 것입니다.

모델의 설정값 조정하기: 하이퍼파라미터 튜닝

모델에는 두 종류의 '설정값'이 있습니다.

구분	파라미터 (Parameter)	하이퍼파라미터 (Hyperparameter)
정의	모델이 학습을 통해 스스로 찾아내는 값	사용자가 학습 시작 전 직접 설정하는 값
예시	회귀 계수(Weight), 편향(Bias)	결정 트리의 최대 깊이, 학습률, 규제 강도
비유	공부하며 깨달은 '문제 풀이 요령'	공부 시간, 사용하는 참고서의 종류

최적의 조합을 찾는 전략

하이퍼파라미터를 어떻게 설정하느냐에 따라 모델의 성능이 천차만별입니다. 이를 효율적으로 찾는 방법이 있습니다.

- 그리드 서치 (Grid Search):** 가능한 모든 조합을 격자(Grid) 형태로 만들어 일일이 대입하는 방식입니다. 꼼꼼하지만 시간이 매우 오래 걸립니다.
- 랜덤 서치 (Random Search):** 정해진 범위 내에서 무작위로 조합을 선택해 시도하는 방식입니다. 모든 조합을 확인하지는 않지만, 훨씬 빠르게 준수한 성능의 조합을 찾을 수 있습니다. 특히 모델 성능에 결정적인 영향을 주는 '중요한 하이퍼파라미터'가 소수일 때, 무작위로 흠뻑려진 점들이 그리드보다 더 세밀하게 최적값을 찾아낼 확률이 높기 때문에 매우 효율적입니다.

Scikit-learn의 `GridSearchCV` 를 사용하면 교차 검증(Cross-Validation)과 그리드 서치를 동시에 수행하여 최적의 하이퍼파라미터를 자동으로 찾아낼 수 있습니다.

실무 활용 사례: 개인화 추천 시스템

넷플릭스나 유튜브 같은 추천 시스템에서는 사용자의 취향을 얼마나 세밀하게 반영할지 결정하는 하이퍼파라미터가 중요합니다. 모델이 너무 단순하면(고편향) 모두에게 인기 있는 영상만 추천하고, 너무 복잡하면(고분산) 사용자의 일시적인 클릭 실수까지 반영해 엉뚱한 영상을 추천합니다. 엔지니어들은 `GridSearchCV` 등을 통해 사용자의 만족도가 가장 높은 최적의 모델 복잡도를 찾아냅니다.

과적합을 막는 안전장치: 규제(Regularization)

모델이 특정 데이터 특성에 너무 과하게 의존하여 가중치(Weight)가 비정상적으로 커지는 것을 막는 기법을 **규제(Regularization)**라고 합니다.

직관적 이해: "손실 함수에 벌금 매기기"

모델은 기본적으로 '오차(Loss)'를 줄이는 방향으로 학습합니다. 규제는 여기에 '**가중치의 크기**'라는 **벌금 항목**을 추가하는 것입니다. $\text{최종 손실} = \text{원래 오차} + \text{규제 항(가중치 크기에 비례)}$ 이제 모델은 오차만 줄이는 것이 아니라, 가중치 값을 너무 크게 키우지 않아야 전체 손실이 줄어듭니다. 즉, 모델이 특정 특성에 과하게 집착하지 못하도록 제약을 거는 것입니다.

기술 설명: L1 규제 vs L2 규제

가중치 w_1, w_2 를 축으로 하는 2차원 평면을 상상해 보세요. 규제는 이 평면 위에 모델이 가질 수 있는 가중치의 범위를 제한하는 '울타리'를 치는 것과 같습니다.

1. L2 규제 (Ridge) - "골고루 나누어 가져라"

- **방법:** 가중치들의 **제곱합**을 더합니다.
- **특징:** 가중치가 커질수록 벌금이 기하급수적으로 늘어납니다. 따라서 가중치들을 전반적으로 작게 유지하며, 특정 특성이 독점하는 것을 막고 여러 특성을 골고루 사용하게 만듭니다.
- **시각적 묘사:** 제약 조건이 원점을 중심으로 하는 **원형**의 형태를 띩니다. 손실 함수의 등고선이 이 원과 만나는 지점에서 최적값이 결정되는데, 원의 곡면 때문에 가중치가 완전히 0이 되기보다는 아주 작은 값으로 유지되는 경향이 있습니다.

2. L1 규제 (Lasso) - "필요 없는 건 버려라"

- **방법:** 가중치들의 **절대값 합**을 더합니다.
- **특징:** 중요하지 않은 특성의 가중치를 **정확히 0**으로 만들어 버립니다. 결과적으로 불필요한 특성을 제거하는 '특성 선택(Feature Selection)' 효과가 있어 모델이 매우 단순해집니다.
- **시각적 묘사:** 제약 조건이 **다이아몬드(마름모)** 형태를 띩니다. 등고선이 이 다이아몬드와 만날 때, 뾰족한 꼭짓점(축 위)에서 만날 확률이 매우 높습니다. 축 위에서 만난다는 것은 다른 가중치가 0이 된다는 뜻이며, 이것이 Lasso가 특성을 선택하는 이유입니다.

[그림 삽입: L1 규제의 다이아몬드 형태 제약 영역과 L2 규제의 원형 제약 영역이 가중치 평면에서 손실 함수 등고선과 만나는 지점을 보여주는 도식]

실무 활용 사례: 금융권 신용 점수 예측

은행에서 대출 심사를 위한 신용 점수 모델을 만들 때, 수백 개의 변수(소득, 직업, 거주지, 소비 패턴 등)를 사용합니다. 이때 특정 변수 하나에 가중치가 너무 쏠리면, 그 변수 하나만으로 당락이 결정되는 불안정한 모델이 됩니다. 이때 **L2 규제**를 적용해 가중치를 분산시킴으로써 모델의 안정성을 높이거나, **L1 규제**를 통해 수백 개의 변수 중 실제로 신용도에 영향이 없는 불필요한 변수들을 제거하여 모델의 해석력을 높입니다.

실습: 최적의 모델을 완성하는 튜닝 프로세스

이번 실습에서는 '가짜 특성(Noise Features)'이 섞인 데이터를 생성하여, L1(Lasso) 규제가 어떻게 불필요한 특성을 제거하고 L2(Ridge) 규제가 어떻게 가중치를 조절하는지 직접 확인해 보겠습니다.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV, learning_curve
from sklearn.linear_model import Ridge, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error

# 1. 데이터 생성: 10개의 특성 중 3개만 유효하고 7개는 가짜(Noise)인 데이터
np.random.seed(42)
n_samples = 100
n_features = 10

X = np.random.randn(n_samples, n_features)
# 실제 정답(y)에 영향을 주는 가중치 (앞의 3개만 값이 있고 나머지는 0)
true_weights = np.array([10, -5, 2, 0, 0, 0, 0, 0, 0, 0])
y = X @ true_weights + np.random.normal(0, 1, n_samples)

# 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 스케일링 (규제 모델 사용 시 필수)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 2. 하이퍼파라미터 튜닝 (Ridge vs Lasso 비교)
param_grid = {'alpha': [0.01, 0.1, 1, 10, 100]}

# Ridge 튜닝
ridge_grid = GridSearchCV(Ridge(), param_grid, cv=5, scoring='neg_mean_squared_error')
ridge_grid.fit(X_train_scaled, y_train)
best_ridge = ridge_grid.best_estimator_

# Lasso 튜닝
lasso_grid = GridSearchCV(Lasso(), param_grid, cv=5, scoring='neg_mean_squared_error')
lasso_grid.fit(X_train_scaled, y_train)
best_lasso = lasso_grid.best_estimator_

print(f"Ridge 최적 alpha: {ridge_grid.best_params_}")
print(f"Lasso 최적 alpha: {lasso_grid.best_params_}")

# 3. 가중치(Weight) 비교 시각화
plt.figure(figsize=(10, 5))
plt.plot(best_ridge.coef_, 'o-', label='Ridge Weights', markerfacecolor='none')
plt.plot(best_lasso.coef_, 's-', label='Lasso Weights', markerfacecolor='none')
plt.axhline(0, color='black', linestyle='--', linewidth=1)
plt.title("Comparison of Model Weights (Ridge vs Lasso)")
plt.xlabel("Feature Index")
plt.ylabel("Weight Value")
plt.legend()
plt.grid(True)
plt.show()

# 4. 학습 곡선(Learning Curve) 시각화 함수
def plot_learning_curve(model, X, y, title):
    train_sizes, train_scores, test_scores = learning_curve(
        model, X, y, cv=5, scoring='neg_mean_squared_error',
        train_sizes=np.linspace(0.1, 1.0, 5))

    plt.plot(train_sizes, -train_scores.mean(axis=1), 'o-', label='Training score')
    plt.plot(train_sizes, -test_scores.mean(axis=1), 'o-', label='Cross-validation score')
    plt.title(title)
    plt.xlabel("Training Set Size")
    plt.ylabel("MSE")

```



```
plt.legend()

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plot_learning_curve(best_ridge, X_train_scaled, y_train, "Ridge Learning Curve")
plt.subplot(1, 2, 2)
plot_learning_curve(best_lasso, X_train_scaled, y_train, "Lasso Learning Curve")
plt.show()

# 5. 최종 평가
y_pred_ridge = best_ridge.predict(X_test_scaled)
y_pred_lasso = best_lasso.predict(X_test_scaled)
print(f"Ridge 테스트 MSE: {mean_squared_error(y_test, y_pred_ridge):.4f}")
print(f"Lasso 테스트 MSE: {mean_squared_error(y_test, y_pred_lasso):.4f}")
```

실행 결과 해설

1. **가중치(Weight) 그래프 해석 (핵심!)**: - **Lasso(Lasso Weights)**: 그래프를 보면 유효한 특성 (0, 1, 2번)의 가중치는 유지되지만, 가짜 특성(3~9번)의 가중치가 **정확히 0**에 수렴하는 것을 볼 수 있습니다. 이것이 바로 Lasso의 '**특성 선택**' 효과입니다. - **Ridge(Ridge Weights)**: 가중치들이 전반적으로 작아지긴 했지만, 0이 되지는 않고 작은 값들을 유지하며 골고루 퍼져 있습니다.
2. **학습 곡선 해석**: - 훈련 오차와 검증 오차의 간격이 좁으면서 함께 낮게 수렴한다면, 하이퍼파라미터 튜닝을 통해 과적합을 성공적으로 방지하고 일반화 성능을 확보한 것입니다.
3. **MSE 결과**: - 이번 실습 데이터는 `true_weights`를 설정할 때 10개의 특성 중 3개만 유효하게 설정한 **희소(Sparse) 데이터셋**입니다. 따라서 불필요한 특성을 과감히 제거하는 Lasso가 가중치를 골고루 나누는 Ridge보다 더 좋은 성능(낮은 MSE)을 보일 가능성이 높습니다.

초보자가 자주 하는 실수: '데이터 누수'와 '검증 데이터 오염'

가장 위험한 실수는 "테스트 데이터를 보고 하이퍼파라미터를 수정하는 것"입니다. 이를 **데이터 누수(Data Leakage)**라고 합니다.

데이터 누수란?

테스트 데이터는 모델이 태어나서 처음 보는 '미지의 데이터'여야 합니다. 그런데 테스트 데이터의 결과를 보고 "음, 성능이 낮네? 하이퍼파라미터를 좀 바꿔볼까?"라고 수정한다면, 결과적으로 **테스트 데이터의 정보가 모델 설계에 반영**된 셈이 됩니다.

이는 마치 기말고사 시험지를 미리 훑쳐보고 공부한 뒤 시험을 치는 것과 같아, 실제 현장에 배포했을 때 성능이 처참하게 떨어지는 결과를 초래합니다.

올바른 파이프라인

반드시 다음 순서를 지켜주세요. 데이터 분리(Train/Val/Test) $\rightarrow$ Train으로 학습 $\rightarrow$ Val로 튜닝(Grid Search) $\rightarrow$ 최종 모델 결정 $\rightarrow$ Test로 단 한 번 평가

확인 문제

Q1. 모델이 훈련 데이터에 대해서는 매우 높은 성능을 보이지만, 새로운 데이터에 대해서는 성능이 급격히 떨어지는 현상을 무엇이라고 하나요? () 과소적합 () 과적합 () 일반화 () 데이터 누수

Q2. 편향-분산 트레이드오프에 대한 설명으로 옳은 것은 무엇인가요? ① 모델이 복잡해질수록 편향은 커지고 분산은 작아진다. ② 모델이 단순해질수록 분산은 커지고 편향은 작아진다. ③ 편향과 분산을 동시에 최소화하는 최적의 모델 복잡도를 찾는 것이 중요하다. ④ 편향과 분산은 서로 독립적이어서 각각 따로 최적화할 수 있다.

Q3. L1 규제(Lasso)와 L2 규제(Ridge)의 차이점으로 가장 적절한 것은 무엇인가요? ① L1은 가중치를 전반적으로 작게 만들고, L2는 중요하지 않은 가중치를 0으로 만든다. ② L1은 중요하지 않은 가중치를 0으로 만들어 특성 선택 효과가 있으며, L2는 가중치를 전반적으로 작게 유지한다. ③ L1은 원형의 제약 조건을 가지고, L2는 다이아몬드 형태의 제약 조건을 가진다. ④ L2 규제는 L1 규제보다 모델을 더 단순하게 만드는 경향이 있다.

Q4. 하이퍼파라미터 튜닝 과정에서 '데이터 누수(Data Leakage)'를 방지하기 위한 가장 올바른 방법은 무엇인가요? ① 테스트 데이터셋을 사용하여 GridSearchCV 를 수행한다. ② 훈련 데이터셋만 사용하여 모델을 학습시키고 평가까지 마친다. ③ 테스트 데이터셋은 모든 튜닝이 끝난 후 최종 성능 평가를 위해 단 한 번만 사용한다. ④ 검증 데이터셋 없이 훈련 데이터셋의 정확도만 보고 하이퍼파라미터를 결정한다.

정답 및 해설

A1. 정답: 과적합 - 해설: 학습 데이터의 소음까지 학습하여 새로운 데이터에 대한 응용력(일반화 성능)이 떨어진 상태를 과적합이라고 합니다.

A2. 정답: ③ - 해설: 모델이 복잡해지면 편향은 낮아지지만 분산이 커지고, 단순해지면 분산은 낮아지지만 편향이 커집니다. 따라서 두 합이 최소가 되는 지점을 찾는 것이 핵심입니다.

A3. 정답: ② - 해설: L1(Lasso)은 절대값 합을 규제하여 가중치를 0으로 만드는 특성이 있고, L2(Ridge)는 제곱합을 규제하여 가중치를 전반적으로 작게 억제합니다.

A4. 정답: ③ - 해설: 테스트 데이터가 튜닝 과정에 개입되면 모델이 테스트 데이터에 '오염'되어 실제 성능보다 높게 측정되는 데이터 누수가 발생합니다. 따라서 테스트 세트는 최종 평가용으로만 격리해야 합니다.

핵심 요약 및 미니 프로젝트

핵심 요약 (Key-Takeaways)

- **과적합(Overfitting)**: 학습 데이터의 노이즈까지 학습해 일반화 성능이 떨어진 상태 $\rightarrow$ 규제와 튜닝으로 해결.
- **과소적합(Underfitting)**: 모델이 너무 단순해 패턴을 못 찾은 상태 $\rightarrow$ 모델 복잡도 증가 필요.
- **편향-분산 트레이드오프**: $\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$. 두 값의 합이 최소가 되는 지점이 최적의 모델.
- **하이퍼파라미터 튜닝**: GridSearchCV 나 RandomSearch 를 통해 검증 데이터셋 기준 최적의 설정값을 탐색.
- **규제(Regularization)**: 가중치에 벌금을 부여해 과적합 방지. **L1(Lasso)**은 특성 선택 효과, **L2(Ridge)**는 가중치 분산 효과.
- **데이터 나누기**: Train $\rightarrow$ Validation $\rightarrow$ Test 순서의 엄격한 분리 준수.

미니 프로젝트: 나만의 최적 모델 완성하기

목표: 기존에 진행했던 '집값 예측' 또는 '스팸 분류' 프로젝트에 본 장의 내용을 적용해 성능을 끌어올리세요.

체크리스트: - [] 데이터를 Train / Validation / Test 세트로 엄격히 분리했는가? - [] GridSearchCV 를 사용하여 최적의 하이퍼파라미터를 찾았는가? - [] L1(Lasso) 또는 L2(Ridge) 규제를 적용해 보았는가? - [] 학습 곡선(Learning Curve)을 그려 과적합/과소적합 여부를 진단했는가? - [] 최종 테스트 세트로 평가한 성능이 훈련 세트 성능과 큰 차이가 없는가?

결과 보고서 작성 가이드: 단순히 "성능이 좋아졌다"가 아니라, 아래와 같은 **성능 비교 표**를 작성하여 객관적으로 증명하세요.

평가 지표	튜닝 전 (기본 모델)	튜닝 후 (규제+튜닝 모델)	개선 정도
훈련 데이터 MSE	12.5	15.2	$\uparrow$ (약간 상승)
테스트 데이터 MSE	45.8	18.4	$\downarrow$ 59.8% 개선
결정계수 (R^2)	0.62	0.88	$\uparrow$ 0.26 상승

- **분석 포인트**: 훈련 데이터 성능은 약간 떨어졌더라도, 테스트 데이터 성능이 크게 올랐다면 이는 과적합을 성공적으로 해결하고 **일반화 성능**을 확보했다는 증거입니다.
- **첨부 자료**: 선택된 최적 하이퍼파라미터 값과 튜닝 전/후의 학습 곡선 그래프를 함께 첨부하세요.

12. 앙상블과 실전 머신러닝

이번 장에서 배우는 내용

단일 모델 하나만으로는 해결하기 어려운 복잡한 문제들이 있습니다. 이번 장에서는 여러 개의 모델을 결합해 더 강력한 예측력을 만들어내는 '앙상블(Ensemble)' 기법을 배웁니다.

- **앙상블 학습의 기본 원리**: 왜 여러 모델을 합치면 성능이 올라가는지 이해합니다.
- **배깅(Bagging)과 랜덤 포레스트**: 무작위 샘플링을 통해 모델의 분산을 줄이고 과적합을 방지하는 방법을 배웁니다.
- **부스팅(Boosting)과 XGBoost**: 잔차 학습을 통해 틀린 문제에 집중하며 성능을 끌어올리는 순차적 학습 방식을 배웁니다.
- **실전 구현 및 비교**: 단일 모델과 앙상블 모델의 성능 차이를 직접 확인하고 경향성을 분석합니다.
- **특성 중요도 분석**: 모델이 어떤 데이터를 중요하게 생각했는지 분석하여 데이터 인사이트를 도출하는 방법을 익힙니다.

집단지성의 힘: 앙상블 학습의 개념

실생활 비유: 전문가 위원회의 결정

어떤 어려운 문제를 해결해야 한다고 가정해 봅시다. 한 명의 천재적인 전문가에게 답을 묻는 것도 좋지만, 때로는 여러 명의 평범한 전문가들이 모여 토론하고 다수결로 결정하는 것이 훨씬 더 정확할 때가 많습니다. 한 사람이 가진 편견이나 실수(오류)를 다른 사람들이 보완해주기 때문입니다.

개념 설명

머신러닝에서도 이와 같은 '집단지성'의 원리를 적용한 것이 바로 **앙상블(Ensemble)**입니다. 앙상블은 하나의 강력한 모델을 만드는 대신, 여러 개의 '약한 학습기(Weak Learner)'를 조합하여 하나의 '강한 학습기(Strong Learner)'를 만드는 기법입니다.

기술 설명

앙상블의 핵심 아이디어는 **다양성(Diversity)**과 **결합(Combination)**입니다.

1. **다양성**: 모든 모델이 똑같이 예측한다면 합치는 의미가 없습니다. 서로 다른 데이터를 학습하거나, 서로 다른 알고리즘을 사용하여 모델들이 각기 다른 관점에서 데이터를 바라보게 해야 합니다.

2. **결합**: 이렇게 만들어진 여러 모델의 예측 결과를 하나로 합칩니다. 분류 문제에서는 '다수결(Voting)' 방식을, 회귀 문제에서는 '평균(Averaging)' 방식을 주로 사용합니다.

배깅(Bagging): 무작위로 샘플링하여 분산을 줄여 과적합을 방지하는 방법

실생활 비유: 서로 다른 참고서를 본 학생들

시험 공부를 할 때, 한 권의 완벽한 교과서만 판 학생은 그 교과서에 나온 특이한 유형의 문제에 과하게 집착할 수 있습니다. 이를 머신러닝에서는 '과적합'이라고 합니다. 반면, 여러 명의 학생이 각자 조금씩 다른 참고서를 공부한 뒤 서로 의견을 교환한다면, 특정 참고서의 오류나 편향에 빠지지 않고 더 일반적인 정답을 찾아낼 수 있을 것입니다.

개념 설명

배깅(Bagging)은 'Bootstrap Aggregating'의 줄임말입니다. 전체 데이터셋에서 무작위로 샘플을 뽑아 여러 개의 작은 데이터셋을 만들고, 각각의 데이터셋으로 모델을 학습시킨 뒤 그 결과를 합치는 방식입니다.

여기서 중요한 개념이 '**편향-분산 트레이드오프**'입니다. - **편향(Bias)**: 모델이 너무 단순해서 데이터의 본질적인 패턴을 잡지 못하는 상태 (과소적합) - **분산(Variance)**: 모델이 너무 복잡해서 훈련 데이터의 작은 노이즈까지 학습해버리는 상태 (과적합)

배깅의 핵심 목적은 바로 이 **분산(Variance)**을 줄이는 것입니다. 여러 모델의 예측값을 평균 내어 특정 데이터에 지나치게 민감하게 반응하는 것을 막는 원리입니다.

기술 설명

- **부트스트랩(Bootstrap) 샘플링**: 중복을 허용하여 무작위로 데이터를 추출하는 방식입니다. 이를 통해 각 모델은 전체 데이터의 조금씩 다른 부분을 학습하게 됩니다.
- **랜덤 포레스트(Random Forest)**: 배깅의 가장 대표적인 알고리즘입니다. 수많은 **결정 트리(Decision Tree)**를 만들고, 그 트리들의 예측 결과를 다수결로 통합합니다.
 - 단순히 데이터만 무작위로 뽑는 것이 아니라, 트리를 만들 때 사용할 **특성(Feature)**까지 무작위로 선택하여 모델 간의 다양성을 극대화합니다.
- **핵심 효과**: 특정 데이터나 특성에 지나치게 의존하는 것을 막아주므로, 단일 결정 트리의 최대 약점인 **과적합(Overfitting)**을 효과적으로 방지합니다.

부스팅(Boosting): 틀린 문제에 집중하여 점수를 올리는 방법

실생활 비유: 오답 노트 작성법

공부를 잘하는 학생들의 비결 중 하나는 '오답 노트'입니다. 처음 문제를 풀고 틀린 문제들을 따로 모아, 왜 틀렸는지 집중적으로 분석하고 다시 공부합니다. 이렇게 '부족한 부분'을 계속해서 보완해 나가면 결국 모든 문제를 맞힐 수 있게 됩니다.

개념 설명

부스팅(Boosting)은 배깅처럼 동시에 여러 모델을 만드는 것이 아니라, **순차적(Sequential)**으로 모델을 만듭니다. 첫 번째 모델이 틀린 데이터에 대해 가중치를 높여, 두 번째 모델이 그 틀린 부분을 더 잘 맞추도록 학습시키는 방식입니다. 즉, 배깅이 분산을 줄이는 데 집중한다면, 부스팅은 **편향(Bias)**을 줄여 **예측 성능을 극대화**하는 데 집중합니다.

기술 설명

- **순차적 학습과 잔차(Residual)**: $\text{모델 1} \rightarrow \text{모델 2} \rightarrow \text{모델 3} \dots$ 순으로 학습합니다. 핵심은 앞선 모델이 예측한 값과 실제 정답 사이의 차이인 '**잔차(Residual)**'를 다음 모델의 학습 목표로 삼는 것입니다.
 - **예시**: 실제 집값이 100인데 모델 1이 80으로 예측했다면, 잔차는 20이 됩니다. 이때 모델 2는 '100'을 맞추는 것이 아니라, 모델 1이 놓친 '**20이라는 오차를 예측하도록** 학습합니다. 이렇게 앞선 모델들이 해결하지 못한 '남은 오차'를 다음 모델이 계속해서 메꾸는 방식입니다.
- **XGBoost (Extreme Gradient Boosting)**: 현재 실무와 캐글(Kaggle) 같은 경진대회에서 가장 사랑받는 부스팅 알고리즘입니다.
 - **정규화(Regularization)**: 모델이 학습 데이터에 너무 과하게 집착하여 복잡해지지 않도록 가중치에 '페널티'를 주는 장치입니다. 구체적으로 **L1, L2 정규화** 기법을 사용하여 불필요한 가지를 쳐내고(Pruning) 모델을 단순하게 유지함으로써, 부스팅의 고질적인 문제인 과적합을 효과적으로 억제합니다.
 - 병렬 처리 최적화가 잘 되어 있어 학습 속도가 매우 빠릅니다.
- **배깅 vs 부스팅 차이점**

구분	배깅 (Bagging)	부스팅 (Boosting)
학습 방식	병렬적 (동시에 여러 모델 학습)	순차적 (앞선 모델의 오차/잔차 보완)
목적	분산 감소 $\rightarrow$ 과적합 방지	편향 감소 $\rightarrow$ 예측 성능 극대화
특징	안정적이며 구현이 쉬움	매우 강력하지만 과적합 위험이 있음
대표 모델	랜덤 포레스트 (Random Forest)	XGBoost, LightGBM, CatBoost

실전! 앙상블 모델로 성능 끌어올리기

이제 실제로 단일 결정 트리와 앙상블 모델(랜덤 포레스트, XGBoost)의 성능을 비교해 보겠습니다. 유방암 진단 데이터셋을 활용하여 암 여부를 예측하는 분류기를 만들어 보겠습니다.

파이썬 코드 실습

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

# 1. 데이터 로드 및 준비
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 2. 모델 정의
models = {
    "단일 결정 트리": DecisionTreeClassifier(random_state=42),
    "랜덤 포레스트 (Bagging)": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost (Boosting)": XGBClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
}

# 3. 모델 학습 및 성능 평가
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    results[name] = acc
    print(f"{name} 정확도: {acc:.4f}")

# 4. 결과 비교 시각화
print("\n--- 최종 성능 비교 ---")
for name, acc in results.items():
    print(f"{name}: {acc:.4f}")
```

실행 결과 해설

코드를 실행하면 환경이나 무작위 시드(random_state)에 따라 수치에 차이가 있을 수 있으나, 일반적인 경향성은 다음과 같습니다.

- **단일 결정 트리 (약 93% 내외)** $\rightarrow$ 학습 데이터에 너무 최적화되어 테스트 데이터에서는 성능이 상대적으로 낮게 나타나는 경향이 있습니다.
- **랜덤 포레스트 (약 96% 내외)** $\rightarrow$ 여러 트리의 의견을 종합하여 단일 트리보다 훨씬 안정적이고 일반화 성능이 높게 나타납니다.

- **XGBoost (약 97% 내외)** $\rightarrow$ 잔차를 학습하며 정밀하게 보완해 나가므로 가장 높은 예측력을 보여주는 경우가 많습니다.

결론적으로, 단일 모델보다는 앙상블 모델이 데이터의 복잡한 패턴을 더 잘 잡아내며 월등한 예측 성능을 보여주는 경향이 있습니다.

앙상블 모델의 해석력과 중요도

앙상블 모델의 큰 장점 중 하나는 '어떤 특성이 예측에 중요하게 작용했는가'를 수치로 알려준다는 점입니다. 이를 **특성 중요도(Feature Importance)**라고 합니다.

단일 트리는 가지가 뻗어 나가는 모양을 직접 그려서 볼 수 있지만, 앙상블 모델은 수백 개의 트리가 섞여 있어 그림으로 볼 수 없습니다. 대신, 각 특성이 불순도를 얼마나 많이 낮췄는지를 수치화하여 제공합니다.

특성 중요도 분석 실습 코드

```
# XGBoost 모델의 특성 중요도 추출 및 시각화
xgb_model = XGBClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
xgb_model.fit(X_train, y_train)

# 특성 중요도 가져오기
importances = xgb_model.feature_importances_
feature_df = pd.DataFrame({'Feature': X.columns, 'Importance': importances})
feature_df = feature_df.sort_values(by='Importance', ascending=False).head(5)

# 시각화
plt.figure(figsize=(10, 6))
plt.barh(feature_df['Feature'], feature_df['Importance'], color='skyblue')
plt.xlabel('Importance')
plt.title('Top 5 Important Features (XGBoost)')
plt.gca().invert_yaxis()
plt.show()

print(feature_df)
```

데이터 인사이트 도출하기

이 수치는 단순히 모델의 성능을 확인하는 것을 넘어 매우 중요한 비즈니스 가치를 제공합니다. 우리는 이 분석을 통해 "암 진단에 있어서 어떤 지표(예: 최악 면적, 최악 반지름 등)가 가장 결정적인 요인이었는가"를 정량적으로 파악할 수 있습니다.

단순히 예측만 하는 것이 아니라, 어떤 변수가 결과에 결정적인 영향을 주었는지 분석함으로써 **도메인 지식을 발견**하고, 이를 바탕으로 구체적인 **비즈니스 전략**이나 **의료 가이드라인**을 세우는 등 실질적인 인사이트를 얻을 수 있습니다.

실무 활용 사례

앙상블 모델은 높은 예측력 덕분에 실제 산업 현장에서 매우 광범위하게 사용됩니다.

1. **랜덤 포레스트 (금융권 신용 평가)**: 고객의 소득, 직업, 기존 대출 이력 등 수많은 변수를 종합하여 대출 승인 여부를 결정합니다. 특정 변수 하나에 휘둘리지 않고 안정적인 판단을 내려야 하는 신용 평가 모델에 적합합니다.
2. **XGBoost (이커머스 고객 이탈 예측)**: 고객의 접속 빈도, 구매 주기, 장바구니 담기 횟수 등을 분석해 이탈 가능성이 높은 고객을 정밀하게 찾아냅니다. 아주 작은 패턴의 차이로 성능이 갈리는 타겟 마케팅이나 추천 시스템의 랭킹 모델에서 주로 사용됩니다.

앙상블 모델 사용 시 주의사항과 실수

성능이 좋다고 해서 무조건 앙상블 모델만 사용하는 것이 정답은 아닙니다.

1. **연산 비용의 증가**: 모델 하나를 돌릴 때보다 수백 개의 모델을 돌려야 하므로 학습 시간과 메모리 사용량이 크게 증가합니다. 실시간 응답이 중요한 시스템에서는 부담이 될 수 있습니다.
2. **블랙박스(Black Box) 특성**: 모델이 '왜' 그런 결과를 내놓았는지 논리적으로 설명하기가 매우 어렵습니다. 의료나 금융처럼 '설명 가능성'이 중요한 분야에서는 단일 트리나 선형 모델이 더 선호될 수 있습니다.
3. **부스팅 모델의 과적합 위험**: 부스팅은 오답(잔차)에 계속 집중하기 때문에, 학습 데이터의 노이즈(이상치)까지 학습해버릴 위험이 있습니다. 따라서 `learning_rate` (학습률)나 `max_depth` (트리 깊이) 같은 하이퍼파라미터 튜닝에 매우 민감하므로 주의가 필요합니다.

핵심 요약

- 앙상블은 여러 약한 모델을 결합해 강한 모델을 만드는 집단지성 기법이다.
- 배깅(Bagging)은 무작위 샘플링을 통해 분산을 줄여 과적합을 방지하며, 대표적으로 랜덤 포레스트가 있다.
- 부스팅(Boosting)은 잔차를 학습하며 순차적으로 오차를 보완해 편향을 줄이고 성능을 극대화하며, 대표적으로 XGBoost가 있다.
- 앙상블 모델은 성능은 뛰어나지만 연산 비용이 높고 해석력이 낮아지는 트레이드-오프가 존재한다.

확인 문제

1. 배깅과 부스팅의 가장 근본적인 학습 방식의 차이는 무엇인가요?
2. 랜덤 포레스트가 단일 결정 트리보다 과적합에 강한 이유는 무엇인가요?

3. XGBoost 모델을 사용할 때, 학습 데이터에 너무 과하게 최적화되었다면 어떤 조치를 취해야 할까요?

정답 및 해설 1. 배깅은 여러 모델을 독립적으로 동시에 학습시키는 병렬 방식이고, 부스팅은 이전 모델의 오차(잔차)를 보완하며 순차적으로 학습시키는 방식입니다. 2. 부트스트랩 샘플링과 특성 무작위 선택을 통해 모델 간의 다양성을 확보하고, 이들의 결과를 평균/다수결로 합쳐 특정 데이터에 대한 의존도(분산)를 낮추기 때문입니다. 3. 학습률(learning_rate)을 낮추거나, 트리의 최대 깊이(max_depth)를 제한하거나, 내장된 정규화 파라미터를 조정하여 모델의 복잡도를 낮춰야 합니다.

미니 프로젝트: 우리 동네 집값 예측 앙상블 모델 만들기

목표: Scikit-learn에서 제공하는 **California Housing dataset**을 활용하여 단일 결정 트리, 랜덤 포레스트, XGBoost 모델의 예측 성능(RMSE)을 비교하고, 집값에 가장 큰 영향을 주는 상위 5개 특성을 찾아 리포트를 작성해 보세요.

가이드라인 및 실습 팁: 1. **데이터 로드:** `from sklearn.datasets import fetch_california_housing` 를 사용하여 데이터를 불러오세요. 2. **전처리:** 데이터를 훈련 세트와 테스트 세트로 분리하세요. 3. **모델 학습:** - `DecisionTreeRegressor` - `RandomForestRegressor` - `XGBRegressor` 세 가지 모델을 각각 학습시키세요. 4. **성능 평가:** `mean_squared_error` 를 사용하여 RMSE(Root Mean Squared Error)를 계산하고 어떤 모델이 가장 오차가 적은지 비교하세요. 5. **분석:** `feature_importances_` 를 통해 집값에 가장 큰 영향을 주는 변수(예: 소득 수준, 집의 연식 등) 상위 5개를 시각화하세요.

시작 코드 예시:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from xgboost import XGBRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor

# 데이터 로드
housing = fetch_california_housing()
X, y = housing.data, housing.target

# 이후 과정은 위에서 배운 파이프라인(분리 -> 학습 -> 평가 -> 분석)을 적용해 보세요!
```

13. 실전 프로젝트 - 타이타닉 생존자 예측

이번 장에서 배우는 내용

지금까지 우리는 머신러닝의 개별 도구들을 하나씩 배웠습니다. 데이터 전처리법, 다양한 모델의 원리, 그리고 모델을 평가하는 방법까지 말이죠. 하지만 실제 현장에서는 이 도구들을 따로 사용하지 않습니다. 데이터 수집부터 최종 예측까지 하나의 유기적인 흐름, 즉 '**파이프라인(Pipeline)**'으로 연결하여 사용합니다.

이번 장에서는 머신러닝의 전 과정을 직접 경험해보기 위해, 데이터 분석의 고전이자 입문 코스인 '**타이타닉 생존자 예측 프로젝트**'를 수행합니다. 단순히 코드를 따라 치는 것이 아니라, 데이터 분석가의 관점에서 가설을 세우고 이를 검증하며 모델을 완성하는 전체 워크플로우를 학습하겠습니다.

실생활 비유: 데이터 분석가는 '디지털 탐정'이다

머신러닝 프로젝트를 수행하는 과정은 마치 **사건 현장을 조사하는 탐정**의 업무와 비슷합니다.

1. **현장 조사(EDA)**: 사건 현장에 도착해 흩어진 증거물들을 살펴봅니다. "피해자는 누구인가?", "주변에 어떤 흔적이 남아 있는가?"를 파악하며 사건의 윤곽을 잡습니다.
2. **증거 정제(전처리)**: 흩이 묻은 지문이나 찢어진 메모지를 깨끗하게 정리합니다. 쓸모없는 정보는 버리고, 중요한 단서를 명확하게 만듭니다.
3. **추리 모델 구축(학습)**: 수집된 증거들을 바탕으로 "범인은 이런 특성을 가진 사람일 것이다"라는 가설을 세우고 논리를 구성합니다.
4. **검증 및 추론(평가)**: 세운 가설이 실제 사실과 맞는지 대조해보고, 틀린 부분이 있다면 추리 과정을 수정합니다.

우리는 이제 타이타닉이라는 거대한 비극의 데이터를 통해, "과연 어떤 조건의 사람들이 생존 확률이 높았을까?"라는 수수께끼를 푸는 디지털 탐정이 되어볼 것입니다.

1. 프로젝트 개요 및 데이터 탐색 (EDA)

가장 먼저 할 일은 우리가 다룰 데이터가 어떻게 생겼는지 확인하는 것입니다. 타이타닉 데이터셋에는 승객의 티켓 등급, 성별, 나이, 함께 탄 가족 수 등 다양한 정보가 담겨 있습니다.

데이터 구조 파악하기

단순히 데이터를 불러오는 것에 그치지 않고, `info()` 와 `describe()` 를 통해 데이터의 '건강 상태'를 체크해야 합니다.

```
import pandas as pd
import seaborn as sns

# 타이타닉 데이터셋 로드 (seaborn 라이브러리 제공)
df = sns.load_dataset('titanic')

# 1. 데이터의 전반적인 구조 확인
print("--- 데이터 기본 정보 ---")
print(df.info())

# 2. 수치형 데이터의 통계적 특성 확인
print("\n--- 수치형 데이터 통계 ---")
print(df.describe())

# 3. 결측치 확인
print("\n--- 결측치 개수 ---")
print(df.isnull().sum())
```

[실행 결과 해설] * `df.info()` 를 통해 `age`, `embarked`, `deck` 컬럼에 결측치(NaN)가 많다는 것을 알 수 있습니다. 특히 `deck` 은 결측치가 너무 많아 분석에서 제외하는 것이 좋을 것 같습니다. * `df.describe()` 를 보면 나이(`age`)의 평균은 약 29세이며, 요금(`fare`)의 편차가 매우 크다는 점을 발견할 수 있습니다. 이는 일부 초부유층 승객이 있었음을 시사합니다.

탐정의 관점: 여기서 우리는 가설을 세웁니다. "당시 사회적 분위기가 '여성'과 '아이'가 먼저 구조되었을 것이고, '1등석 승객'이 생존율이 더 높았을 것이다." 이 가설이 맞는지 확인하는 것이 이번 프로젝트의 핵심입니다.

2. 데이터 전처리 및 피처 엔지니어링

머신러닝 모델은 숫자만 이해할 수 있는 '계산기'입니다. 따라서 텍스트로 된 데이터를 숫자로 바꾸고, 빈칸(결측치)을 적절히 채워주는 과정이 필요합니다. 사실 모델 선택보다 더 중요한 것이 바로 이 전처리 과정입니다.

데이터 누수(Data Leakage) 방지하기

전처리를 할 때 가장 주의해야 할 점은 '미래의 정보'를 미리 가져다 쓰지 않는 것입니다. 전체 데이터의 중앙값으로 결측치를 채운 뒤 데이터를 나누면, 학습 데이터에 테스트 데이터의 정보가 스며드는 '데이터 누수'가 발생합니다. 따라서 반드시 데이터를 먼저 분리한 후, 학습 데이터의 통계량만을 사용하여 전처리해야 합니다.

인코딩의 선택: Label vs One-Hot

텍스트를 숫자로 바꿀 때, `LabelEncoder` 는 단순히 0, 1, 2와 같은 순서대로 숫자를 부여합니다. 하지만 모델은 숫자 2가 0보다 '크다' 혹은 '중요하다'라고 오해할 수 있습니다. * **Label Encoding**: 성별(남/여)처럼 두 가지 값만 있는 경우 효율적입니다. * **One-Hot Encoding**: 승선항(S, C, Q)처럼 여러 값이고 순서가 없는 경우, 각각의 값을 독립된 컬럼으로 만들어 가중치 왜곡을 방지하는 것이 더 적합합니다. (본 예제에서는 단순화를 위해 `LabelEncoder`를 사용하지만, 실무에서는 `One-Hot Encoding`을 권장합니다.)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# 1. 분석에 불필요한 컬럼 제거 및 특성 생성
df['family_size'] = df['sibsp'] + df['parch'] + 1
drop_cols = ['deck', 'embark_town', 'alive', 'who', 'adult_male', 'class']
df_cleaned = df.drop(columns=drop_cols)

# 2. 특성(X)과 정답(y) 분리
X = df_cleaned.drop('survived', axis=1)
y = df_cleaned['survived']

# 3. [중요] 데이터 분리 먼저 수행 (데이터 누수 방지)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. 학습 데이터의 통계량으로 결측치 처리
age_median = X_train['age'].median()
embarked_mode = X_train['embarked'].mode()[0]

X_train['age'] = X_train['age'].fillna(age_median)
X_test['age'] = X_test['age'].fillna(age_median) # 학습 데이터의 중앙값을 그대로 사용

X_train['embarked'] = X_train['embarked'].fillna(embarked_mode)
X_test['embarked'] = X_test['embarked'].fillna(embarked_mode)

# 5. 범주형 데이터 인코딩
le = LabelEncoder()
for col in ['sex', 'embarked']:
    # 학습 데이터로 fit 하고, 학습/테스트 데이터 모두 transform
    X_train[col] = le.fit_transform(X_train[col])
    X_test[col] = le.transform(X_test[col])

print("전처리가 완료되었습니다. 데이터 누수 없이 분리되었습니다.")
```

3. 모델 선택 및 학습

이제 준비된 데이터를 모델에게 학습시킬 차례입니다. 이번 프로젝트는 '생존(1)이나 사망(0)이나'를 맞추는 **분류(Classification)** 문제입니다.

우리는 여러 개의 결정 트리를 만들어 다수결로 결과를 내는 **랜덤 포레스트(Random Forest)** 모델을 선택하겠습니다. 이 모델은 표 형식의 데이터에서 매우 강력한 성능을 발휘하며, 과적합 위험이 적습니다.

```
from sklearn.ensemble import RandomForestClassifier

# 모델 생성 및 학습
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

print("모델 학습이 완료되었습니다.")
```

4. 모델 평가 및 최적화

단순히 "정확도가 80%다"라고 말하는 것은 부족합니다. 모델이 어떤 데이터를 헛갈려 하는지 분석해야 합니다.

오차 행렬(Confusion Matrix) 분석

오차 행렬을 보면 '살았는데 죽었다고 예측한 경우(FP)'와 '죽었는데 살았다고 예측한 경우(FN)'를 구분해 볼 수 있습니다.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 예측 수행
y_pred = model.predict(X_test)

# 결과 출력
print(f"정확도: {accuracy_score(y_test, y_pred):.4f}")
print("\n--- 오차 행렬 ---")
print(confusion_matrix(y_test, y_pred))
print("\n--- 상세 보고서 ---")
print(classification_report(y_test, y_pred))
```

어떤 변수가 중요했을까? (Feature Importance)

랜덤 포레스트의 장점은 모델이 판단을 내릴 때 어떤 변수를 중요하게 생각했는지 알려준다는 점입니다.

```
import matplotlib.pyplot as plt
import numpy as np

# 변수 중요도 추출
importances = model.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize=(10, 6))
plt.title('Feature Importances')
plt.barh(range(len(indices)), importances[indices], align='center')
plt.yticks(range(len(indices)), [X.columns[i] for i in indices])
plt.xlabel('Relative Importance')
plt.show()
```

[해설] 보통 sex 와 fare (또는 pclass)가 가장 높은 중요도를 보입니다. 이는 우리의 가설("여성과 부유층이 생존 확률이 높다")이 데이터로 입증되었음을 의미합니다.

모델 최적화 (GridSearchCV)

더 높은 성능을 위해 하이퍼파라미터를 튜닝합니다. `GridSearchCV` 를 사용해 최적의 `max_depth` 와 `n_estimators` 조합을 찾습니다.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5)
grid_search.fit(X_train, y_train)

print(f"최적 파라미터: {grid_search.best_params_}")
print(f"최적 모델 정확도: {grid_search.best_score_:.4f}")
```

5. 실무 활용 사례: 이진 분류 모델의 확장

타이타닉 생존자 예측은 전형적인 '이진 분류(Binary Classification)' 문제입니다. 실무에서 이러한 구조의 모델은 매우 광범위하게 사용됩니다.

- **고객 이탈 예측 (Churn Prediction):** 고객의 이용 패턴(접속 빈도, 결제 금액 등) 데이터를 분석하여, 이 고객이 서비스를 해지할 것인지(1) 유지할 것인지(0)를 예측합니다. 이를 통해 이탈 가능성이 높은 고객에게만 맞춤 쿠폰을 발송하는 전략을 세울 수 있습니다.
- **금융 신용 등급 평가 (Credit Scoring):** 대출 신청자의 소득, 직업, 기존 연체 기록 등을 분석하여 대출금을 상환할 수 있을지(1) 없을지(0)를 판단하여 대출 승인 여부를 결정합니다.
- **질병 진단 (Medical Diagnosis):** 환자의 검사 수치, 증상 데이터를 바탕으로 특정 질병이 양성(1)인지 음성(0)인지를 판별하는 보조 도구로 활용됩니다.

6. 프로젝트 마무리: 인사이트 도출 및 회고

최종 결과 해석

우리는 타이타닉 데이터를 통해 다음과 같은 결론을 얻었습니다. 1. **성별(Sex)**은 생존을 결정짓는 가장 압도적인 요인이었습니다. 2. **객실 등급(Pclass)**과 **요금(Fare)** 또한 중요한 변수였으며, 이는 사회적 지위가 생존율에 영향을 미쳤음을 보여줍니다. 3. **가족 규모(FamilySize)**는 너무 적거나 너무 많을 때보다 적당한 규모일 때 생존율에 긍정적인 영향을 주는 경향이 있었습니다.

머신러닝 파이프라인 흐름도

이번 프로젝트를 통해 경험한 전체 흐름은 다음과 같습니다.

[데이터 수집] $\rightarrow$ [EDA(가설 설정)] $\rightarrow$ [데이터 분리] $\rightarrow$ [전처리(결측치/인코딩)] $\rightarrow$ [특성 생성] $\rightarrow$ [모델 학습] $\rightarrow$ [평가(오차 행렬)] $\rightarrow$ [최적화(튜닝)] $\rightarrow$ [인사이트 도출]

자주 하는 실수

- **데이터 누수(Data Leakage):** 전처리를 할 때 `X_test`의 정보를 미리 사용하여 `X_train`의 결측치를 채우는 실수를 합니다. 반드시 학습 데이터의 통계량(중앙값 등)만 사용하여 테스트 데이터를 처리해야 합니다.
- **정확도 맹신:** 정확도가 높다고 해서 좋은 모델은 아닙니다. 생존자와 사망자의 비율이 불균형하다면, 정밀도(Precision)와 재현율(Recall)을 함께 살펴봐야 합니다.

확인 문제

1. 데이터 전처리 과정에서 `train_test_split`을 수행하기 전에 전체 데이터의 중앙값으로 결측치를 채우면 어떤 문제가 발생하나요?
2. `LabelEncoder`를 사용할 때, 범주가 3개 이상인 데이터(예: 승선항)에 적용할 경우 발생할 수 있는 위험성은 무엇인가요?
3. 모델의 성능을 평가할 때 '정확도(Accuracy)'만으로 판단하기 어려운 이유는 무엇인가요?

정답 및 해설

1. **정답:** 데이터 누수(Data Leakage)가 발생합니다. 테스트 데이터의 정보가 학습 과정에 포함되어, 실제 환경보다 성능이 과하게 높게 측정되는 '낙관적 편향'이 생길 수 있습니다.
2. **정답:** 모델이 숫자의 크기($0 < 1 < 2$)를 순서나 가중치로 인식하여, 실제로는 관계가 없는 범주 간에 서열이 있다고 잘못 판단할 수 있습니다. 이를 방지하기 위해 원-핫 인코딩(One-Hot Encoding)을 사용합니다.
3. **정답:** 데이터 불균형 문제 때문입니다. 예를 들어 생존자가 10%뿐인 데이터에서 모두 '사망'이라고만 예측해도 정확도는 90%가 나오지만, 정작 중요한 '생존자'는 한 명도 맞추지 못한 쓸모 없는 모델이 될 수 있습니다.

미니 프로젝트: 나만의 생존 예측 파이프라인 구축하기

목표: 지금까지 배운 과정을 스스로 구현하여, 데이터 전처리부터 평가까지의 전체 파이프라인을 구축해 보세요.

과제: 1. `seaborn`의 `titanic` 데이터셋을 다시 로드하세요. 2. **새로운 가설 세우기:** "나이가 아주 어린거나(아동) 아주 많은(노인) 사람이 더 많이 생존했을 것이다"라는 가설을 검증하기 위해 `is_child` 또

는 `is_senior` 라는 새로운 특성을 만들어 보세요. 3. **파이프라인 구현:** * 데이터 분리 $\rightarrow$ 결측치 처리(학습 데이터 기준) $\rightarrow$ 인코딩 $\rightarrow$ 모델 학습 $\rightarrow$ 평가 순으로 코드를 작성하세요. 4. **결과 분석:** 새로 만든 특성이 `feature_importances_` 에서 얼마나 중요한 비중을 차지하는지 확인하고, 자신의 가설이 맞았는지 결론을 내리세요.

14. 실전 프로젝트 - 집값 예측

이번 장에서 배우는 내용

이번 장에서는 지금까지 배운 회귀 분석의 모든 기술을 집약하여, 실제 부동산 데이터를 활용한 '**집값 예측 파이프라인**'을 구축합니다. 단순히 모델에 데이터를 넣고 결과를 확인하는 것이 아니라, 데이터 탐색(EDA)부터 도메인 지식을 반영한 특성 공학, 그리고 모델의 한계를 분석하는 잔차 분석까지의 전 과정을 실습합니다.

- 부동산 데이터의 특성과 다변량 회귀의 이해
 - 데이터 시각화를 통한 가설 설정 및 이상치 제거
 - 모델의 성능을 극대화하는 특성 공학(Feature Engineering)
 - 다양한 회귀 모델 비교 및 Scikit-learn 파이프라인 구축
 - RMSE, R^2 및 잔차 분석을 통한 정밀한 모델 평가
 - 변수 중요도 확인 및 하이퍼파라미터 튜닝을 통한 개선
-

실생활 비유: 베테랑 공인중개사의 감정 평가

여러분 주변에 수십 년 경력의 베테랑 공인중개사가 있다고 가정해 봅시다. 이분은 집값을 매길 때 단순히 "평수가 넓으니 비싸다"라고 말하지 않습니다.

1. **데이터 탐색**: 먼저 주변 시세를 살피고, 최근 거래된 비슷한 집들의 가격을 확인합니다. (EDA)
2. **특성 추출**: "이 집은 지은 지는 오래됐지만, 최근에 리모델링을 했으니 가치가 높겠군", "역세권이라 직장인 수요가 많겠어"라며 단순한 수치 너머의 의미를 찾아냅니다. (특성 공학)
3. **예측**: 자신이 가진 경험(학습된 모델)을 바탕으로 적정 가격을 제시합니다. (모델링)
4. **피드백**: 만약 예측가보다 훨씬 높게 팔렸다면, "내가 놓친 호재가 있었나?"라며 분석합니다. (잔차 분석 및 모델 개선)

머신러닝의 회귀 파이프라인은 바로 이 '**베테랑 중개사의 사고 과정**'을 코드로 구현하는 것과 같습니다.

부동산 데이터의 특성과 예측의 본질

집값 예측은 머신러닝에서 **다변량 회귀(Multivariate Regression)**의 가장 전형적인 사례입니다. 집값이라는 하나의 결과값(종속 변수)에 영향을 주는 요소(독립 변수)가 매우 다양하기 때문입니다.

종속 변수와 독립 변수의 관계

- **종속 변수(Target)**: 우리가 예측하려는 값인 Price (집값)입니다.
- **독립 변수(Features)**: 집값에 영향을 주는 방 개수, 면적, 위치, 건축 연도, 범죄율 등이 있습니다.

데이터의 왜곡(Skewness) 확인

실제 부동산 데이터를 보면, 대부분의 집은 평균적인 가격대에 몰려 있지만, 극소수의 초고가 펜트하우스들이 가격 분포를 오른쪽으로 길게 늘어뜨리는 경향이 있습니다. 이를 '**오른쪽 꼬리가 긴 분포 (Right-Skewed)**'라고 합니다.

이런 왜곡이 심하면 모델은 극단적인 고가 주택에 영향을 받아 전체적인 예측선을 과도하게 위로 끌어올리게 되고, 결과적으로 일반적인 집값들에 대한 예측력이 떨어지는 '편향'이 발생합니다. 따라서 우리는 데이터를 정규분포 형태로 만들어주는 작업이 필요합니다.

데이터 탐색 및 가설 설정 (EDA)

데이터를 무작정 모델에 넣기 전에, 어떤 변수가 정말 중요한지 '눈으로 확인'하는 과정이 필요합니다.

1. 상관관계 분석 (Heatmap)

어떤 변수가 집값과 강한 상관관계를 갖는지 확인하기 위해 상관관계 행렬을 그리고 히트맵으로 시각화합니다. 예를 들어, '면적'과 '집값'의 상관관계수가 0.8이라면 매우 강한 양의 상관관계가 있다고 볼 수 있습니다.

2. 선형성 확인 (Scatter plot)

상관계수가 높더라도 실제로 직선의 형태를 띠는지 산점도로 확인해야 합니다. 만약 곡선 형태라면 단순 선형 회귀보다는 더 복잡한 모델이나 데이터 변환이 필요하다는 가설을 세울 수 있습니다.

3. 이상치(Outlier) 제거 전략

면적은 매우 좁은데 가격이 말도 안 되게 높거나, 반대로 면적은 엄청나게 넓은데 가격이 매우 낮은 데이터는 모델의 학습을 방해하는 '노이즈'가 됩니다. 이러한 이상치는 제거하거나 보정하는 전략을 세워야 합니다.

특성 공학: 데이터의 가치를 높이는 과정

특성 공학(Feature Engineering)은 원본 데이터를 모델이 학습하기 좋은 형태로 가공하는 과정입니다. 여기서 **도메인 지식**이 빛을 발합니다.

1. 범주형 데이터의 인코딩

컴퓨터는 '강남구', '서초구'라는 글자를 이해하지 못합니다. 이를 0, 1, 2로 바꾸는 레이블 인코딩보다는, 각 구를 독립된 열로 만들어 0 또는 1로 표시하는 원-핫 인코딩(One-Hot Encoding)을 사용하여 특정 지역이 갖는 가중치를 독립적으로 학습하게 합니다.

2. 로그 변환 (Log Transformation)

앞서 언급한 '왜곡된 분포'를 해결하는 방법입니다. 가격에 로그를 취하면 초고가 주택과 같은 극단적인 값(Outlier)의 영향력을 줄여 모델의 편향을 막을 수 있습니다. 큰 값들은 상대적으로 작게 압축되고, 작은 값들 간의 차이는 유지되어 분포가 정규분포에 가까워지므로 모델의 안정성이 크게 향상됩니다.

3. 파생 변수 생성

단순히 건축 연도를 넣는 것보다, 현재 연도 - 건축 연도를 계산해 **집의 나이(House Age)**라는 변수를 만드는 것이 훨씬 직관적입니다. 모델 입장에서 "1990년에 지어졌다"보다 "34년 된 집이다"라는 정보가 예측에 더 유리하기 때문입니다.

4. 스케일링 (Scaling)

면적은 수천 단위이고, 방 개수는 1~5단위입니다. 단위 차이가 너무 크면 모델이 면적 변수에만 과도하게 반응할 수 있습니다. 이를 방지하기 위해 모든 수치를 일정 범위로 맞추는 **StandardScaler**나 **MinMaxScaler**를 적용합니다.

파이썬 코드 실습: 집값 예측 파이프라인

이 실습에서는 가상의 부동산 데이터를 생성하여 선형 회귀와 랜덤 포레스트 모델의 성능을 비교하고, 잔차 분석까지 수행하는 전체 파이프라인을 구현해 보겠습니다.

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. 가상 데이터 생성
np.random.seed(42)
n_samples = 1000
data = {
    'Size': np.random.normal(100, 30, n_samples), # 면적
    'Rooms': np.random.randint(1, 6, n_samples), # 방 개수
    'Age': np.random.randint(0, 50, n_samples), # 집의 나이
    'Location': np.random.choice(['City', 'Suburb', 'Rural'], n_samples), # 지역
    'Price': 0 # 타겟 변수
}
df = pd.DataFrame(data)
# 집값 결정 로직: 면적*1000 - 나이*500 + 지역가중치 + 노이즈 (비선형성 추가)
loc_map = {'City': 50000, 'Suburb': 20000, 'Rural': 0}
df['Price'] = df['Size']*1000 - (df['Age']**1.5)*100 + df['Location'].map(loc_map) +
np.random.normal(0, 5000, n_samples)

# 2. 전처리: 타겟 변수 로그 변환 (왜곡 및 편향 방지)
df['Log_Price'] = np.log1p(df['Price'])

# 3. 특성 및 타겟 분리
X = df[['Size', 'Rooms', 'Age', 'Location']]
y = df['Log_Price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. 전처리 파이프라인 설정
numeric_features = ['Size', 'Rooms', 'Age']
categorical_features = ['Location']
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(), categorical_features)
    ])

# 5. 모델 비교 분석 (Linear Regression vs Random Forest)
models = [
    ('Linear Regression', LinearRegression()),
    ('Random Forest', RandomForestRegressor(n_estimators=100, random_state=42))
]

results = {}
plt.figure(figsize=(15, 5))

for i, (name, model) in enumerate(models):
    # 파이프라인 구축 및 학습
    pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('regressor', model)])
    pipeline.fit(X_train, y_train)

    # 예측 및 역변환
    y_pred_log = pipeline.predict(X_test)
    y_test_exp = np.exp1(y_test)
    y_pred_exp = np.exp1(y_pred_log)

```

```

# 지표 계산
mae = mean_absolute_error(y_test_exp, y_pred_exp)
rmse = np.sqrt(mean_squared_error(y_test_exp, y_pred_exp))
r2 = r2_score(y_test_exp, y_pred_exp)
results[name] = {'MAE': mae, 'RMSE': rmse, 'R2': r2}

# 실제값 vs 예측값 시각화
plt.subplot(1, 2, i+1)
plt.scatter(y_test_exp, y_pred_exp, alpha=0.5)
plt.plot([y_test_exp.min(), y_test_exp.max()], [y_test_exp.min(), y_test_exp.max()], 'r--')
plt.title(f'{name}: Actual vs Predicted')
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')

plt.tight_layout()
plt.show()

# 결과 출력
print(pd.DataFrame(results).T)

# 6. 잔차 분석 (최종 선택된 Random Forest 기준)
final_pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('regressor',
RandomForestRegressor(n_estimators=100, random_state=42))])
final_pipeline.fit(X_train, y_train)
y_pred_final_log = final_pipeline.predict(X_test)
y_pred_final_exp = np.expml(y_pred_final_log)
y_test_exp = np.expml(y_test)

residuals = y_test_exp - y_pred_final_exp

plt.figure(figsize=(8, 6))
plt.scatter(y_pred_final_exp, residuals, alpha=0.5)
plt.axhline(y=0, color='r', linestyle='--')
plt.title('Residual Plot (Random Forest)')
plt.xlabel('Predicted Price')
plt.ylabel('Residuals (Actual - Predicted)')
plt.show()

```

실행 결과 해설

1. 모델 비교 분석 - Linear Regression: 데이터가 완벽한 직선 관계일 때 유리합니다. 하지만 실제 집값은 나이의 제공에 비례하거나 지역별 특성이 복잡하게 얹혀 있어 R^2 점수가 상대적으로 낮게 나올 수 있습니다. - **Random Forest:** 트리 기반 모델은 데이터의 비선형적인 관계(예: 나이가 많아 질수록 가격이 급격히 떨어지는 구간 등)를 훨씬 더 잘 잡아냅니다. 따라서 일반적으로 선형 회귀보다 MAE와 RMSE가 낮고 R^2 가 높게 나타납니다.

2. 성능 지표 해석 - MAE / RMSE: 실제 가격과 예측 가격의 평균적인 오차입니다. RMSE가 MAE보다 훨씬 크다면, 일부 매물에서 예측이 크게 빗나갔음을 의미합니다. - **R^2 Score:** 1에 가까울수록 모델이 집값의 변동성을 잘 설명하고 있다는 뜻입니다.

3. 잔차 분석(Residual Plot) 해석 - 그래프 확인법: x축(예측값)에 따라 y축(잔차)의 점들이 0을 중심으로 위아래로 무작위하게(Randomly) 퍼져 있는지 확인해야 합니다. - **정상:** 특정 패턴 없이 고르게 퍼져 있다면, 모델이 데이터의 주요 패턴을 모두 학습했고 남은 오차는 단순한 '노이즈'라는 뜻입니다. - **비정상:** 만약 잔차가 U자나 역U자 형태의 곡선을 그린다면, 모델이 아직 잡아내지 못한 비선형

패턴이 남아 있다는 신호입니다. 이때는 특성 공학을 통해 새로운 변수를 추가하거나 더 정교한 모델을 검토해야 합니다.

모델 성능 평가 및 결과 해석

회귀 모델에서는 단순히 "정확도가 몇 %"라고 말할 수 없습니다. 따라서 더 정밀한 분석이 필요합니다.

잔차 분석의 실무적 의미

잔차 분석은 모델의 '사각지대'를 찾는 과정입니다. 예를 들어, 저가 주택에서는 예측이 잘 되는데고가 주택으로 갈수록 잔차가 커진다면, "고가 주택에만 영향을 주는 특수 변수(예: 브랜드 가치, 조망권)가 누락되었다"는 인사이트를 얻을 수 있습니다.

실전적 관점의 모델 개선 및 마무리

모델의 수치만 낮추는 것이 아니라, "왜 이런 결과가 나왔는가"를 분석하는 것이 실무의 핵심입니다.

1. 중요 변수 확인 (Feature Importance)

랜덤 포레스트와 같은 트리 기반 모델은 어떤 변수가 예측에 가장 큰 기여를 했는지 알려줍니다. 만약 `Location`의 중요도가 압도적으로 높다면, 지역별 세부 데이터를 더 보강하여 모델을 개선할 수 있습니다.

2. 하이퍼파라미터 튜닝

`GridSearchCV` 등을 사용하여 결정 트리의 깊이(`max_depth`)나 나무의 개수(`n_estimators`)를 최적화해 과적합을 방지하고 일반화 성능을 높입니다.

3. 비즈니스적 가치 해석

최종적으로 "이 모델을 통해 집값을 예측했을 때 오차 범위가 $\pm 5\%$ 이내이므로, 실제 매물 가격 산정의 기초 자료로 활용 가능하다"라는 비즈니스적 결론을 내리는 것이 프로젝트의 완성입니다.

자주 하는 실수

- **데이터 누수(Data Leakage)**: 전체 데이터에 대해 스케일링을 먼저 하고 데이터를 나누는 실수입니다. 반드시 **학습 데이터로만 스케일러를 학습(`fit`)시키고**, 테스트 데이터에는 적용(`transform`)만 해야 합니다. (이를 위해 위 코드에서 `Pipeline`을 사용했습니다.)

- **로그 변환 후 역변환 망각:** 타겟 변수에 로그를 취했다면, 최종 결과값은 반드시 `np.exp(m1())` 등을 통해 원래 단위로 되돌려야 합니다. 그렇지 않으면 예측값이 수천만 원이 아니라 '15.4' 같은 로그값으로 출력됩니다.
- **과도한 변수 추가:** 무조건 변수를 많이 넣는다고 성능이 좋아지지 않습니다. 서로 상관관계가 너무 높은 변수들이 많으면 '다중공선성' 문제가 발생해 모델이 불안정해질 수 있습니다.

핵심 요약

- **회귀 파이프라인:** EDA $\rightarrow$ 특성 공학 $\rightarrow$ 모델링 $\rightarrow$ 평가 $\rightarrow$ 개선의 반복 과정이다.
- **특성 공학:** 로그 변환으로 극단적 값의 영향력을 줄여 편향을 막고, 원-핫 인코딩으로 범주형 데이터를 처리하며, 도메인 지식으로 파생 변수를 만든다.
- **모델 평가:** MAE, RMSE로 오차의 크기를 측정하고, R^2 로 설명력을 확인하며, 잔차 분석으로 모델의 편향과 패턴 누락을 체크한다.
- **파이프라인:** Pipeline 을 사용하면 전처리부터 예측까지의 과정을 하나로 묶어 데이터 누수를 방지하고 관리를 효율화할 수 있다.

확인 문제

1. 집값 데이터처럼 한쪽으로 치우친 분포에서 초고가 주택과 같은 극단적인 값이 모델의 예측선을 과도하게 끌어올리는 것을 방지하기 위해 사용하는 변환 기법은 무엇인가요?
2. '건축 연도'라는 변수보다 '집의 나이'라는 변수가 모델에 더 유리한 이유는 무엇인가요?
3. 회귀 모델의 잔차 분석 그래프에서 점들이 0을 중심으로 무작위하게 퍼져 있지 않고 특정한 패턴(예: 곡선)을 보인다면, 이는 무엇을 의미하나요?

정답 및 해설 1. **로그 변환(Log Transformation):** 큰 값을 상대적으로 작게 압축하여 분포의 왜곡(Skewness)을 줄이고 모델의 편향을 방지합니다. 2. **직관적인 특성 제공:** 모델이 '현재 시점으로부터 얼마나 오래되었는가'라는 상대적 가치를 더 쉽게 학습할 수 있기 때문입니다. 3. **모델의 패턴 학습 미흡:** 모델이 데이터 내의 특정 비선형 관계나 중요한 변수 간의 상호작용을 제대로 학습하지 못했음을 의미하며, 특성 추가나 모델 변경이 필요합니다.

미니 프로젝트: 나만의 집값 예측 모델 고도화

제공된 코드를 바탕으로 다음 기능을 추가하여 모델을 고도화해 보세요. 1. **새로운 파생 변수 추가:** 예를 들어 `면적 / 방 개수` (방 하나당 평균 면적) 변수를 만들어 성능이 향상되는지 확인하세요. 2. **모델 비교 확장:** XGBoost 나 `GradientBoostingRegressor` 를 추가하여 랜덤 포레스트보다 성능이 더 좋아지는

지 비교해 보세요. 3. **하이퍼파라미터 튜닝**: GridSearchCV 를 사용하여 랜덤 포레스트의 최적 파라미터를 찾아보고, 튜닝 전후의 R^2 점수 차이를 기록해 보세요.

15. 실전 프로젝트 - 영화 추천 시스템
